context-mode 从入门到精通

项目	说明
主题	context-mode —— 面向 AI 编码代理的上下文窗口优化工具（GitHub：`mksglu/context-mode`）
版本基线	context-mode 1.0.169（npm 最新稳定版）
核实日期	2026-10-30（npm registry、官方 README、官方 `skills/context-mode/SKILL.md`、`BENCHMARK.md`）
主线平台	Claude Code（插件市场安装，hooks 与会话延续支持最完整）
适用环境	Windows、macOS、Linux；Node.js >= 22.5 或 Bun
预设读者	已在使用 AI 编码代理（Claude Code、OpenCode、Cursor 等）的开发者；不要求先用过 context-mode，也不要求曾经手写过 hooks

数据口径声明：文中出现的节省比例与字节数（如 315 KB 压缩至 5.4 KB）均来自官方 `BENCHMARK.md` 自测基准，代表官方自身工作负载下的结果，其具体收益因任务类型而异；评估收益时应以 `ctx_stats` 在本机实测为准。无法核实的细节本文不予记载，凡与官方文档存在出入之处，以官方文档为准。

前言

本书面向两类读者：一类是在 Claude Code 中重度使用 MCP（Model Context Protocol，模型上下文协议）工具、苦于上下文快速增长的开发者；另一类是负责团队效能、希望为多个 AI 编码代理统一部署上下文治理方案的技术负责人。全书按「入门、进阶、高级、精通」四篇递进：入门篇完成安装与首次验证；进阶篇掌握六个数据工具的日常用法；高级篇理解检索排名、会话延续、路由强制与安全模型四套内部机制；精通篇沉淀方法论、集成工作流、运维部署与实战案例。

阅读约定如下。

- 命令、参数、配置键、文件路径与工具名均保持英文原文；正文与示例遵循「中文与英文、数字之间加一个空格」的排版惯例。
- 每条命令首次出现时均给出含义与参数解释；示例可直接粘贴使用，并注明预期输出形态。
- 「路由（routing）」指 context-mode 引导或强制代理把可能产生大输出的操作转交给沙箱工具执行的行为；「沙箱（sandbox）」指隔离的子进程执行环境。「教师文件」指官方提供的路由指引文件（如 `AGENTS.md`、`GEMINI.md`），它只能通过提示词影响模型，不能强制拦截。
- 全文以第一篇第一章的定义为准，术语在后文保持同一种译法；完整术语表见附录 B。

第一篇 入门篇

> 本篇解决三个问题：context-mode 解决什么问题（第 1 章）、如何安装与验证（第 2 章）、如何在不写代码项目的前提下跑通第一个完整用例（第 3 章）。完成本篇后，读者应能独立安装 context-mode、确认各组件工作正常，并解读 `ctx_stats` 的输出。

第 1 章 认识 context-mode

在使用 AI 编码代理的过程中，读者可能遇到这样的情形：让代理分析一份访问日志或抓取一个接口之后，仅仅几次交互，模型的可用上下文就明显缩水，此前的指令开始失效；或者对话进行到一半被系统自动压缩，代理忘记了自己正在编辑哪些文件、有哪些任务尚未完成。本章解释这两类现象在 MCP 使用中的技术成因，介绍 context-mode 针对它们提出的四条解决思路，并说明该工具在同类方案中的定位，为后续安装与使用建立概念基础。

1.1 上下文窗口的流失来自两侧

上下文窗口（context window）是模型单次推理所能读取的全部信息总量，包含系统提示、历史对话、工具输出与模型自身的回复。传统工具链在这里存在两个方向的浪费。

输入侧：工具原始数据整块进入上下文。 MCP 工具的返回值会原样写入对话历史。官方给出的一组典型占用是：一次 Playwright 页面快照约 56 KB，二十个 GitHub issue 约 59 KB，一份服务器访问日志约 45 KB。按照官方测算，密集使用 MCP 工具约 30 分钟后，上下文的约 40% 已经被原始数据占据。

输出侧：模型把 token 花在填充与冗长解释上。 模型的每一次回复同样写入上下文，客套语、逐字复述与过长的说明都会消耗预算。

压缩阶段：历史被清空，状态随之丢失。 当上下文接近上限时，代理系统会执行上下文压缩（compaction），即丢弃或压缩较早的对话内容以腾出空间。压缩之后，模型通常失去三类关键信息：正在编辑的文件、正在进行中的任务，以及使用者最近一次提出的要求。

1.2 context-mode 的定义

context-mode 是一个部署在 MCP 协议层的开源工具：对使用者而言，它表现为一个 MCP 服务器（多数平台可直接注册）或一个平台插件（如 Claude Code 插件）；对模型而言，它额外提供一组以 `ctx_` 为前缀的工具。它的职责可以概括为一句话：把大体积的工具输出截留在沙箱中，只在上下文中留下最终结论；同时把工作状态记录在本地数据库，在上下文压缩后自动恢复。

按官方基准（`BENCHMARK.md`，2026-10-30 版本核实），一次完整会话中 315 KB 的原始工具输出可被压缩至 5.4 KB，会话可用时长从约 30 分钟延长到约 3 小时。该数据为官方自测口径，个体收益取决于任务类型。

项目以 Elastic License 2.0（ELv2）授权发布，属于 source-available 许可：可以自由使用、修改与再分发，但不得将其包装为托管或受管服务出售，也不得移除许可声明。截至核实日，该项目在 GitHub 上处于活跃维护状态。

1.3 四条解决思路

解决思路	具体做法	对应的浪费
Context Saving（上下文节省）	六个 `ctx_` 沙箱工具接管会产生大输出的命令、文件分析与网页抓取，只把 `stdout` 摘要放回上下文	输入侧原始数据
Session Continuity（会话延续）	由钩子（hook，宿主在工具调用前后触发的拦截回调）把文件编辑、git 操作、任务、错误与使用者裁决逐条写入本地 SQLite 数据库；压缩前生成快照，压缩后重建工作状态	压缩导致的失忆
Think in Code（以代码思考）	要求模型写一段脚本完成统计、过滤等分析运算，只把 `console.log` 的结果送回上下文；一段脚本取代十次工具调用	输入侧与输出侧两侧
不干预文风（no prose-style enforcement）	路由块只规定数据流向（哪些数据留在沙箱），不约束模型最终回复的行文、详略与格式	输出侧冗余，同时避免强压缩行文方案损伤编码与推理质量

其中第四条值得单独说明：官方的理由是，激进要求模型精简回复的提示词方案已被证明会劣化编码与推理类基准表现，因此 context-mode 只负责「数据去哪里」，把「话怎么说」留给使用者自己的规范文件（如 `CLAUDE.md`、`AGENTS.md`）。

1.4 定位与边界

从架构上看，context-mode 不是命令行输出过滤器，也不是托管在云端的用量仪表盘；它工作在 MCP 协议层，原始数据在隔离的子进程内处理完毕，从未进入上下文，也从未离开本机。项目不采集遥测、不做云同步、不需要注册账号，所有数据以本地 SQLite 数据库形式存放（托管型 Insight 分析页面是唯一的浏览器端服务，见 11.6 节）。

适用边界同样明确：它是配合宿主（Claude Code、OpenCode、Cursor 等 AI 编码代理）运行的能力增强层，不能独立完成编码任务，也不能替代宿主自身的权限与沙箱机制；涉及任意代码执行的审批语义见 11.5 节。

1.5 全书路线

第二篇以六个数据工具为主线安排进阶内容（第 4 至 7 章）；第三篇解剖四套机制（第 8 至 12 章）；第四篇给出方法论、集成与实战（第 13 至 17 章）。对只想快速上手的读者，完成第 2 章与第 3 章即可开始日常使用；机制细节可需要时再读。

第 2 章 安装与验证

安装是使用 context-mode 的第一步，也是最容易遇到问题的环节：不同平台的安装方式差异较大，缺一个配置项都可能表现为「工具出现了但路由不生效」。本章以 Claude Code 为主线完成端到端安装，逐项解释每个配置的含义，并给出其余平台的安装速查与补偿措施。

2.1 环境要求

项目	要求	说明
Claude Code	v1.0.33 及以上（插件市场路径要求）	在终端执行 `claude --version` 检查；若不识别 `/plugin` 命令，先用 `brew upgrade claude-code` 或 `npm update -g @anthropic-ai/claude-code` 升级
Node.js	>= 22.5，或 Bun	MCP-only 安装与 Claude Code 以外的平台需要；Node.js >= 22.5 时将自动使用内置的 `node:sqlite` 模块
Windows	支持	`better-sqlite3` 原生模块缺失时，安装脚本与运行时钩子会自动重取预编译制品并修复（Issue #408），通常无需手动 `npm rebuild`
Linux	不支持在 Node.js < 22.5 上安装	安装会直接失败并给出修复指引（Issue #564）；Bun 也可用
CentOS/RHEL/Alpine	旧 glibc 与 musl 系统需编译链	见 2.6 节末尾的构建前置说明

在此定义一个贯穿全书的新概念：钩子（hook）。宿主在固定时点（工具调用前、调用后、会话开始、上下文压缩前等）触发的一类拦截回调，回调进程可以读取调用信息、阻断调用或注入内容。context-mode 的程序化拦截能力全部由 hooks 提供。

2.2 在 Claude Code 中安装（推荐路径）

以下两条命令在 Claude Code 会话中输入，用于把插件市场指向本项目的仓库，并从该市场安装插件：

```
/plugin marketplace add mksglu/context-mode
/plugin install context-mode@context-mode
```

完成后重启 Claude Code，或在会话内执行 `/reload-plugins`。

随后执行一次官方诊断命令完成验证：

```
/context-mode:ctx-doctor
```

预期的输出形态：每一项检查都以 `[x]` 显示为通过。ctx_doctor（诊断器）会校验运行时、hooks、FTS5 与注册状态四类内容，任何一项显示未通过，请跳转 17.3 节的故障排除表对照处理。

说明一个对后续理解很重要的行为：插件路径下，路由指令由会话启动钩子在运行时注入，不在项目目录中写入任何文件。历史上早期版本曾自动把教师文件写入项目目录，该行为因会污染 git 工作树而被移除（Issue #158、#164），如今只有少数无 hooks 平台仍需手动复制教师文件（见 2.6 节）。

2.3 安装后得到了什么

插件路径一共注册六类 hooks、十一个 MCP 工具与七个斜杠命令，各自的角色如下表。

六类 hooks

钩子	触发时点	职责
PreToolUse	工具执行前	拦截可能产生大输出的调用，强制改道到沙箱工具（路由强制执行的主关卡）
PostToolUse	工具执行后	抽取结构化事件（文件、任务、git、错误等）写入本地会话数据库
UserPromptSubmit	使用者提交消息时	捕获使用者请求与裁决（例如「改用 X，不要 Y」）
PreCompact	上下文压缩前	生成分优先级的会话快照存库，供压缩后恢复
SessionStart	会话开始时	注入路由指引，并在压缩恢复或续接会话时回放工作状态
Stop	模型回合结束时	记录回合结束时的状态，补充会话时间线

十一个 MCP 工具

前六个是数据处理工具（第二篇逐章展开），后五个是元工具（运维类）：

- 沙箱执行类：`ctx_execute`（在沙箱中运行代码）、`ctx_execute_file`（在沙箱中处理文件）、`ctx_batch_execute`（一次调用批量执行多命令与多检索）
- 知识库类：`ctx_index`（索引入库）、`ctx_search`（检索已索引内容）、`ctx_fetch_and_index`（抓取网页并索引）

- 元工具：`ctx_stats`（查看节省统计）、`ctx_doctor`（诊断安装状态）、`ctx_upgrade`（升级到最新版本并修复 hooks）、`ctx_purge`（永久删除知识库内容）、`ctx_insight`（在浏览器打开托管分析页面）

七个斜杠命令（在 Claude Code 会话中输入）：`/context-mode:ctx-stats`、`/context-mode:ctx-doctor`、`/context-mode:ctx-index`、`/context-mode:ctx-search`、`/context-mode:ctx-upgrade`、`/context-mode:ctx-purge`、`/context-mode:ctx-insight`。斜杠命令是 Claude Code 的插件特性；在其他平台，直接在对话中输入 `ctx stats`、`ctx doctor` 等短命令，模型会自动改调对应的 MCP 工具。

2.4 可选：状态栏

Claude Code 的插件清单无法声明状态栏，需要使用者手动编辑一次 `~/.claude/settings.json`（全局设置），加入：

```
{
  "statusLine": {
    "type": "command",
    "command": "context-mode statusline"
  }
}
```

保存并重启后，状态栏实时显示三类累计值：本次会话节省的金额、跨会话累计节省的金额、效率百分比。该命令通过全局安装的 CLI 解析，与插件缓存位置无关。

2.5 MCP-only 最小安装（试用路径）

若暂不想启用 hooks 与斜杠命令，可只把 context-mode 注册为一个普通 MCP 服务器：

```
claude mcp add context-mode -- npx -y context-mode
```

该路径仍然提供全部十一个 `ctx_` 工具，模型可以自发使用；但由于没有 hooks 拦截，模型不会被程序化地引导优先使用沙箱工具，按照官方对照数据，节省率约 60%，低于启用 hooks 时约 98% 的水平。官方将其定位为「正式启用前的试用配置」。

2.6 其他平台安装速查

非 Claude Code 平台的安装形制可归为三类：插件一步装（Copilot CLI、Codex CLI 的插件路径、`agy`、OMP）、配置文件手工装（多数平台）、MCP-only 装法（所有平台通用）。下表给出要点，逐平台完整步骤见附录 C。

平台	要点	hooks 支持
Gemini CLI	单一 `~/.gemini/settings.json` 同时注册 MCP 与四类 hooks（BeforeTool、AfterTool、PreCompress、SessionStart）；验证命令 `/mcp list` 应显示 `Connected`	是
VS Code Copilot	项目内 `.vscode/mcp.json` 注册服务器，`.github/hooks/context-mode.json` 注册三类 hooks；需 Copilot Chat v0.32+	是
JetBrains Copilot	在 Settings > Tools > AI Assistant > Model Context Protocol 中添加服务器；hooks 文件与 VS Code Copilot 同位于 `.github/hooks/context-mode.json`	是
GitHub Copilot CLI	推荐插件一步装 `copilot plugin install mksglu/context-mode:configs/copilot-cli`；手动路径为 `copilot mcp add` + `~/.copilot/hooks` 下的 hooks 文件	是
Cursor	市场插件待审（Issue #485）；临时用本地目录安装或手工配置 `mcp.json` + `hooks.json`，并复制 `.cursor/rules/context-mode.mdc` 教师文件	部分（`sessionStart` 被其校验器拒绝）
OpenCode	项目或全局 `opencode.json` 中加 `"plugin": ["context-mode"]`；与旧版 `mcp.context-mode` 条目并存会导致零工具注册（v1.0.140+ 有 stderr 提示）	插件内实现
KiloCode	与 OpenCode 同架构，配置文件为 `kilo.json`	插件内实现
Codex CLI	`codex plugin marketplace add mksglu/context-mode`，并在 `[features]` 中开启 `hooks` 与 `plugin_hooks` 后信任插件 hooks；手动路径见附录 C	是
Kimi Code	`~/.kimi-code/mcp.json` 注册 MCP，`~/.kimi-code/config.toml` 以 `[[hooks]]` 表注册六个事件	是
Qwen Code	`~/.qwen/settings.json` 同时注册 MCP 与 hooks；钩子协议与 Claude Code 相同	是
Zed	仅 MCP：`~/.config/zed/settings.json`（Windows 为 `%APPDATA%\Zed\settings.json`）的 `context_servers` 段；需手动复制教师文件	否
Antigravity IDE	仅 MCP：`~/.gemini/antigravity/mcp_config.json`；需手动复制教师文件	否
Antigravity CLI（`agy`）	`agy plugin install https://github.com/mksglu/context-mode/tree/main/configs/antigravity-cli`，要求 `agy` >= 1.0.7	有界支持
Kiro	`.kiro/settings/mcp.json` + `.kiro/hooks/context-mode.json`，并复制 `KIRO.md` 教师文件	是（无会话恢复钩子）
Pi Coding Agent	`pi install npm:context-mode` + `~/.pi/agent/mcp.json`	是（扩展事件）
OMP（Oh My Pi）	`omp plugin install context-mode` 一步装，插件自注册 MCP	是（插件内实现）
OpenClaw 网关	仓库根目录 `npm run install:openclaw`，要求网关版本晚于 2026.1.29	是（原生插件）
Claude Code（对照）	2.2 节插件路径	是（最完整）

特殊系统的构建前置：在 glibc >= 2.31 的系统（Ubuntu 20.04+、Debian 11+、Fedora 34+、macOS、Windows）上，`npm install` 无需构建工具。CentOS 8/RHEL 8 需先安装 `gcc-toolset-10-gcc gcc-toolset-10-gcc-c++ make python3 python3-setuptools` 后再启用工具集安装；CentOS 7/RHEL 7 对应 `devtoolset-10`；Alpine 需要 `build-base python3 py3-setuptools`（better-sqlite3 12.8.0+ 已提供 musl 预编译，多数情况下无需安装）。这几条命令的完整写法见官方 README 的 Build Prerequisites 一节。

2.7 安装验证清单

安装完成后，建议按顺序核对以下五项：

1. 会话内执行 `/context-mode:ctx-doctor`，所有检查项显示 `[x]`。
2. 终端执行 `context-mode doctor`（该命令无需进入会话），确认运行时、hooks、FTS5 与版本四类信息与预期一致。
3. 在会话中输入 `ctx stats`，模型应调用 `ctx_stats` 工具并返回统计页（首次使用时节省值为 0 属正常）。
4. 状态栏（若已配置）开始计数。
5. 选择第 3 章的任一官方示例运行一次，运行后再看 `ctx stats` 确认节省值大于 0。

全部通过即可进入第 3 章；未通过的项目带着 `ctx_doctor` 的输出转第 17 章。

第 3 章 十五分钟上手

不写任何项目代码，也能完整体验 context-mode 的价值：本章使用官方提供的三个开箱场景，依次演示数据型的 `ctx_execute`、统计型的 `ctx_stats` 与长任务型的会话延续。完成本章后，读者应能识别「一次本来会冲进上下文的调用」与「一次被沙箱截留的调用」之间的输出差异。

先回顾两个将反复出现的概念。沙箱（sandbox）：context-mode 为每次代码执行单独启动的隔离子进程，脚本只能通过 `stdout` 把结果送入上下文；stdout：标准输出流，即程序打印到终端的内容，context-mode 只把这一小段文本放入对话。

3.1 第一个沙箱调用

在 Claude Code 会话中，最自然的用法是直接用自然语言下指令，路由（模型自动选择工具的行为）由 hooks 与注入的路由指引共同完成。例如输入：

```
用沙箱统计当前目录中每种扩展名的文件数量，输出一张汇总表，然后运行 ctx stats。
```

模型应调用 `ctx_execute`，在沙箱内运行一段脚本（如遍历目录并计数），最后只把总计行送入上下文；再调用 `ctx_stats` 返回节省统计。预期输出形态是一张几行的小表与一段统计摘要，合计通常不超过 1 KB；若改用普通 Shell 逐个列出文件，同样的目录可能产生数十 KB 的原始列表。

如果想显式控制，也可以直接向模型下达带工具名的指令，下面这条会话指令要求模型调用 `ctx_execute` 并指定语言：

```
用 ctx_execute 以 javascript 执行：读取 package.json，打印 name、version 与 dependencies 数量。
```

`ctx_execute` 的关键参数有两个：`language` 指定后文 4.2 节列出的 12 种语言之一；`code` 为待执行的脚本源码。脚本内应有明确的打印语句，因为只有打印出来的内容才会回到上下文。

3.2 读懂 ctx_stats

`ctx_stats` 是只读的统计工具，不会重置或删除任何内容。输出包含三块：上下文节省（按工具分解的节省量、token 消耗与节省比例）、调用计数（各工具的调用次数）、会话统计（当前会话事件数）。首次使用时数值为 0 指的是「尚未经过路由的调用」，不代表安装异常。与之对照，`ctx_purge` 才会永久删除知识库内容，且需要显式确认（`confirm: true`）。

3.3 场景一：仓库深度调研

官方 Try It 场景的第一条，用于验证批量研究型任务。把下面的提示词整段粘贴到会话中（保留英文原文即可，模型可直接处理）：

```
Research https://github.com/modelcontextprotocol/servers — architecture, tech stack,
top contributors, open issues, and recent activity. Then run /context-mode:ctx-stats.
```

官方口径：本次任务共 5 次调用，进入上下文约 62 KB；若以常规方式把调研中间结果逐页读入上下文，原始数据约为 986 KB，节省约 94%。实现机制是：抓取、解析与交叉汇总都在沙箱中完成，中间页不进入对话。

3.4 场景二：git 历史分析

第二个场景验证「一条命令 + 本地分析」的组合：

```
Clone https://github.com/facebook/react and analyze the last 500 commits:
top contributors, commit frequency by month, and most changed files.
Then run /context-mode:ctx-stats.
```

官方口径：1 次调用，上下文占用约 5.6 KB。克隆与五百条提交记录的统计都在沙箱内完成，进入上下文的只有贡献者、月度频率与改动最多的文件三张小表。与之对照，直接对克隆仓库执行原始 `git log`，仅日志本身即约 11.6 KB（官方基准表中 153 条提交为 11.6 KB，五百条线性放大更多），且不含任何统计结论。

3.5 场景三：会话延续的快速体验

会话延续（Session Continuity）指上下文被压缩后依然能恢复工作状态的能力。官方给出的体验方式：

先发起一个多步任务（例如「创建一个带路由、测试与错误处理的 Express REST API」），持续推进直至工具调用数超过 20 次，然后输入 `ctx stats` 查看会话事件计数——每一次文件编辑、任务创建与用户裁决都已入库。随后让上下文自然触发压缩：压缩完成后，模型会从使用者最后一次提示继续工作，任务清单、已改文件与未解决的错误俱在，无须重复交代。

需要记住的边界行为是：只有通过 `--continue`、`--resume` 或 `/resume` 才会读取上一个会话的存档；若开启一个全新会话且不续接，上一会话的会话数据会被立即删除。这套默认策略把「延续」与「隐私」绑定在一起：续接是明确动作，不续接即清空。

3.6 本章检查点

读者应能回答以下三个问题；任何一个不能回答，请回到对应小节：

1. 为什么模型调用 `ctx_execute` 后，上下文里只剩汇总而不见原始数据？（答案要点：沙箱子进程隔离，只有 stdout 进入上下文。）
2. `ctx_stats` 与 `ctx_purge` 的差别是什么？（答案要点：前者只读统计，后者带确认参数地永久删除。）
3. 什么条件下能看到压缩后的状态恢复？（答案要点：hooks 生效且以续接方式开启会话。）

第二篇 进阶篇

> 本篇围绕六个数据工具展开：沙箱执行 `ctx_execute`（第 4 章）、批量与文件沙箱 `ctx_batch_execute` 与 `ctx_execute_file`（第 5 章）、个人知识库三件套 `ctx_index` 与 `ctx_search` 与 `ctx_fetch_and_index`（第 6 章），以及把它们串成日常习惯的路由规则（第 7 章）。完成本篇后，读者应能独立决定「任何一项会产生大输出的操作」应当交给哪个工具执行，并能让模型在大多数情况下自动做出同样的选择。

第 4 章 用 ctx_execute 把大输出挡在沙箱里

读者也许已经注意到，自己在会话中敲下的多数命令本义并不是「要看到完整输出」：调用接口是为了确认响应是否吻合预期，运行测试是为了知道通过与否，翻 `git log` 是为了找出最近的改动脉络。完整输出与真正需要的结论之间，往往隔着 95% 以上的字节量。本章介绍承担这一转换工作的核心工具 `ctx_execute`，包括其隔离机制、语言选择、常见模式与 Bash 分工。

4.1 隔离机制：只有 stdout 进入上下文

每次调用 `ctx_execute` 时，context-mode 会启动一个隔离子进程：脚本在该进程的独立进程边界内运行，彼此之间无法访问内存或状态。子进程运行脚本、捕获 stdout，随后只有 stdout 这一小段文本进入对话；路径为日志文件、API 响应、页面快照的原始数据在沙箱内经过加工后即被丢弃，从未送入上下文。

由此得出第一条使用纪律：脚本内必须有打印语句。没有 `console.log` 或等价的输出，一次调用等于白耗——上下文什么都得不到。第二条纪律是打印结论而非转储数据：打印出现了哪几个具体异常（含 ID、行号、精确值），而不是把整个响应对象转成字符串再打印一遍，后者只是把洪水换了个名目送回上下文。

4.2 语言与运行时

`ctx_execute` 通过 `language` 参数支持 12 种语言运行时：JavaScript、TypeScript、Python、Shell、Ruby、Go、Rust、PHP、Perl、R、Elixir 与 C#。检测到 Bun 时，JS/TS 的执行速度可提升 3 至 5 倍。

日常绝大多数任务落在下表前三行，其余语言按需选用即可。

任务形态	建议语言	理由
HTTP 调用、JSON 处理	`javascript`	原生 fetch 与 JSON 解析、async/await 顺手
数据分析、CSV、统计	`python`	csv、statistics、collections、re 等标准库齐备
带管道的 Shell 操作、文件模式匹配	`shell`	grep、awk、jq、find、wc、sort、uniq 就地可用

4.3 三个可直接复用的模式

模式一：API 调试（JavaScript）。 以下为官方示例，逻辑是拉取订单接口并进行两类校验后只打印结论：

```
const resp = await fetch('http://localhost:3000/api/orders');
const { orders } = await resp.json();

const bugs = [];
const negQty = orders.filter(o => o.quantity < 0);
if (negQty.length) bugs.push(`Negative qty: ${negQty.map(o => o.id).join(', ')}`);

const nullFields = orders.filter(o => !o.product || !o.customer);
if (nullFields.length) bugs.push(`Null fields: ${nullFields.map(o => o.id).join(', ')}`);

console.log(`${orders.length} orders, ${bugs.length} bugs found:`);
bugs.forEach(b => console.log(`- ${b}`));
```

进入上下文的只有最后几行汇总，例如 `128 orders, 2 bugs found:` 与两条异常明细。

模式二：读取大文件并分析（Python）。 普通读取工具会把整份文件送入上下文；`ctx_execute_file` 则把文件内容预载进沙箱变量 `FILE_CONTENT`（该变量的用法与边界见 5.2 节），`ctx_execute` 也可用等价方式在脚本内自行读取文件路径后分析。

模式三：带统计的命令组合（Shell）。 沙箱中同样可以运行 Shell，官方示例为先跑测试再打印退出码，便于把数千行测试输出折叠成一行结论：

```
npm test 2>&1
echo "EXIT=$?"
```

更强的一类用法是让外部 CLI 在沙箱内先自行收缩输出，例如用 `gh` 的 JSON 查询参数把 PR 列表压成每行一条摘要：

```
gh pr list --json number,title,state,reviewDecision --jq '.[] | "\(.number) [\(.state)] \(.title) — \(.reviewDecision // "no review")"'
```

4.4 intent 意图过滤：超大输出的自动降级

沙箱产出超过 5 KB 且调用时附带 `intent`（意图描述）的情况下，context-mode 会切换到意图驱动过滤：把完整输出索引进本地知识库，再按意图检索出与意图吻合的段落，返回「少量相关片段 + 一组可继续检索的关键词表」。对使用者而言，效果是一个超大的调用结果被自动降级为一小段相关内容，后续需要更多细节时，用量词表继续 `ctx_search` 即可（该流程的参数见第 6 章）。

4.5 凭据透传

带身份验证的 CLI 在沙箱内可直接使用：`gh`、`aws`、`gcloud`、`kubectl`、`docker` 等工具继承宿主进程的环境变量与配置文件路径，凭据本身不会出现在对话里。这解决了「沙箱里还能不能登云」的常见顾虑：鉴权状态由环境继承，模型不需要也拿不到凭据明文。

4.6 与 Bash 的分工

`ctx_execute` 用于一切「读、查、抓、列、测、diff、inspect」类操作；真正适合直接走 Bash 的只是一小批必然小输出的操作。官方路由规则中的 Bash 白名单如下：

类别	示例命令
文件变更	`mkdir`、`mv`、`cp`、`rm`、`touch`、`chmod`
git 写操作	`git add`、`git commit`、`git push`、`git checkout`、`git branch`、`git merge`
导航	`cd`、`pwd`、`which`
进程控制	`kill`、`pkill`
包管理	`npm install`、`npm publish`、`pip install`
简单输出	`echo`、`printf`

白名单之外一律进沙箱。判断口诀同样来自官方规则：拿不准时选 context-mode；上千种会返回数据的 CLI（`gh`、`aws`、`kubectl`、`docker`、`terraform` 等）无法逐个列举，统一按「可能产生大输出」处理。经验数据亦支持这一默认：不路由状态下，一次未加引导的 `curl` 或 Playwright 快照就可能带入约 56 KB，抵销一整个会话的节省。

第 5 章 批量执行与文件沙箱

第 4 章的 `ctx_execute` 一次调用只陪伴一段脚本；当任务需要并行拉取多个数据源、或在一份文件上做一次性提取时，还有两个更匹配形状的工具：批量执行器 `ctx_batch_execute` 与文件沙箱 `ctx_execute_file`。本章说明两者的分工与选型。

5.1 ctx_batch_execute：一次调用聚合多个命令与多个检索

`ctx_batch_execute` 把「多次调用」合并为「一次调用」：在一次请求中同时运行多条命令并执行多个检索查询。对命令型任务（官方口径约为 986 KB 原始数据收敛到 62 KB 的仓库调研场景），聚合调用的价值在于压缩交互轮次：轮次越少，路由拦截与事件捕获的重复开销越少，模型的等待时间也越短。

工具提供可选的 `concurrency` 参数，取值 1 至 8，用于 I/O 密集型批处理（例如并发抓取多个 URL 或调用多个接口）时的并发度。使用者通常不需要手动调用该工具：自然语言指令「一次性并行拉取这些接口并汇总」即可让模型组织批量调用。

5.2 ctx_execute_file：文件留在沙箱内处理

`ctx_execute_file` 把指定路径的文件完整读入沙箱，以预置变量 `FILE_CONTENT` 供脚本使用，进入上下文的依旧只有打印的结论。官方示例（Python）：

```
# FILE_CONTENT 由 ctx_execute_file 预载
import json
data = json.loads(FILE_CONTENT)
print(f"Records: {len(data)}")
```

两个使用要点。其一，适合它的场景是一次性提取：读完即走的结论性分析（统计 JSON 记录数、从日志里抽出错误行、数一数源码里的函数）。其二，若同一份文件会被反复检索多个问题，更经济的路径是先 `ctx_index` 建索引再用 `ctx_search` 多轮问答（见第 6 章），避免重复整块加载。

5.3 工具选型决策表

三个数据执行工具的分工可以压缩为下表。

情形	选用的工具	理由
单段脚本、单接口、单命令组	`ctx_execute`	最通用的形态；12 种语言均可
多命令、多查询、可并行拉取	`ctx_batch_execute`	一次调用聚合多数据源，减少轮次
一份文件的读取与一次性分析	`ctx_execute_file`	文件整块留在沙箱，`FILE_CONTENT` 就地处理
同一文档反复多问	`ctx_index` + `ctx_search`	索引一次，检索多次，每轮只回片段
抓外部网页反复多问	`ctx_fetch_and_index` + `ctx_search`	抓取、转换、索引一步完成并带缓存

5.4 从超大 JSON 中找一条记录

官方 Try It 中有一条刻意构造的压力场景，适合作为 5.3 节选型表的演练：让代理创建一个返回 7.5 MB、两万条记录的本地接口，其中一条隐藏记录位于下标 13000，然后要求「抓取该接口，找出隐藏记录，原样展示内容」。常规路径会把 7.5 MB 响应整体送入上下文；沙箱路径下，脚本在沙箱完成过滤，进入上下文的只有那一条记录。官方口径：原始 7.5 MB，上下文约 0.9 KB，节省约 99%。完整操作步骤见 16.5 节。

第 6 章 个人知识库：ctx_index、ctx_search 与 ctx_fetch_and_index

反复查询同一批资料是常见需求：一份规格说明书、一条长命令的历史输出、一份技术文档，常常要看十次以上，而每次需要的只是其中一段。知识库三件套为此设计：内容入库一次，此后每轮问答只把命中的片段送进上下文。本章按「入库、检索、抓取入库」的顺序讲解参数与策略。

6.1 ctx_index：分块入库

`ctx_index` 将 markdown 内容按标题层切碎为块（分块时保持代码块完整），随后写入本地知识库。底层是 SQLite 的 FTS5 全文索引虚拟表，检索采用 BM25 相关性排序，标题与各级标题在评分中占五倍权重。日志型、纯文本型内容同样可以入库，分块策略对 markdown 结构最优。

两个关键参数需要注意：`path`（文件路径，由服务端在本地读取）与 `content`（内联文本，此时内容会作为工具参数进入上下文）。**红线规则：任何大块数据都应走 `path`，`content` 只用于使用者随手写的小段文本。** 大块数据走 `content` 等于把数据先送进上下文一次（作为工具入参），再入库一次，上下文使用量翻倍。第 14 章会给出该红线在 Playwright 场景中的具体代价对比。

入库时建议同时指定 `source` 名称（例如 `source: "playwright-snap"`、`source: "zod-docs"`），检索时即可按来源过滤；`source` 支持部分匹配，例如 `source: "Node"` 能命中名为 `Node.js v22 CHANGELOG` 的文档。

6.2 ctx_search：按策略检索

`ctx_search` 在一次调用中批量执行多个查询请求，返回围绕命中位置的智能片段（不是整文档，也不是简单截断，见 8.5 节）。要让检索质量稳定，以下三条策略全部来自官方路由规则：

1. **查询词 2 至 4 个、偏技术性名词。**BM25 采用 OR（或）语义，命中越多词的片段自动排得越靠前；宽泛的一两个常用词会让排名模糊。
2. **把所有问题放进一个 `queries` 数组、一次调用问完。**官方明确要求不要拆成多次独立检索调用；例如 `ctx_search(queries: ["transform pipe", "refine superRefine", "coerce codec"], source: "Zod")`。
3. **多来源时必带 `source` 参数**，避免跨来源串味（不同文档相互污染结果）。

渐进节流（涉及版本：1.0.x 全线生效）。 为防止模型以「多查几次」的方式变相把数据库倒进上下文，连续的检索调用会被递进限制：

连续检索调用次数	行为
第 1 至 3 次	正常返回（每次查询返回 2 条结果）
第 4 至 8 次	降额返回（每次查询 1 条）并附警告
第 9 次及以后	直接拦截，并提示改用 `ctx_batch_execute` 聚合

`contentType` 参数可将结果按内容类型过滤，例如只要 `code` 段或只要 `prose`（叙述段）。

6.3 ctx_fetch_and_index：抓网页入库并带 TTL 缓存

`ctx_fetch_and_index` 从 URL 拉取页面，转换为 markdown，分块入库，之后即可用 `ctx_search` 反复提问；原始页面全程不进入上下文。它的另一个价值是缓存：同一 URL 在 TTL（time to live，缓存有效期）窗口内重复调用时直接跳过重新抓取。

事项	行为
默认 TTL	24 小时（可按次以毫秒数覆盖，PR #666 引入 `ttl` 参数）
命中时	返回约 0.3 KB 的缓存提示（对照重新抓取动辄 48 KB 以上的正文进入上游）
过期时	静默重新抓取，无须人工干预
强制刷新	`ttl: 0` 或 `force: true` 绕过缓存强制重抓
内容清理	启动时自动移除超过 14 天的内容库与其来源记录

多 URL 场景下，可传 `requests: [{url, source}, ...]` 数组并配合 `concurrency: 1 至 8` 并行抓取。`ctx_stats` 会单列缓存类指标：命中数、规避的数据量、省下的网络请求数与含缓存的累计节省。缓存内容跨会话保留，因此续接会话（`--continue`）时不会重复抓取文档。

抓取目标优先使用 raw 直链以避免页面包装，例如 GitHub 托管项目的变更日志可直接取 `https://raw.githubusercontent.com/<org>/<repo>/main/CHANGELOG.md`。

6.4 存储位置与清理

知识库内容以按项目的 SQLite 数据库存放在本地磁盘：存储根目录默认随平台适配器（例如 Claude Code 为 `~/.claude/context-mode`），知识库内容位于其下 `content` 子目录，会话数据位于 `sessions` 子目录；根目录可用环境变量 `CONTEXT_MODE_DIR` 覆盖（详见 12.1 节，该变量自 v1.0.147 起提供）。官方部分文档示例中知识库路径写作 `~/.context-mode/content`，以实际环境下 `ctx_doctor` 的输出为准。

需要清空时使用 `ctx_purge` 并显式确认（`confirm: true`），该操作永久删除知识库内全部已索引内容；正式的删除类操作务必核对确认参数，`ctx_stats` 等只读工具不会改动任何数据。

第 7 章 路由规则：让模型默认走沙箱

能熟练使用各个工具，不代表模型在无人引导时也会如此选择：没有系统引导时，模型重拾 Bash 的 `curl`、直接 `cat` 大文件是高频行为，一次未引导的大输出即可冲掉整轮节省。本章给出官方路由规则的全貌，包括 Bash 白名单判断口径、决策树、情形选型总表、自动触发清单与子代理行为，让「默认走沙箱」从纪律变成机制的默认值。

7.1 三层引导如何叠加

让模型优先使用沙箱的力量有三层，强度依次递增：

1. **教学层（教师文件）**：把路由指引文件（`AGENTS.md`、`GEMINI.md`、`context-mode.mdc` 等）放进项目或全局配置，模型按提示词自觉选择工具；档案官方口径约 60% 的场景会照做，但拦不住漏网的调用。
2. **注入层（会话启动注入）**：hooks 在会话开始时把路由指引注入系统提示，同样属提示层面，但使用者无须在项目内放文件。
3. **强制层（hooks 拦截）**：PreToolUse 等钩子在工具执行前程序化拦截、改写或阻断调用，是唯一无法被模型「忘记」的一层，官方口径约 98% 的场景按路由执行。

Claude Code 插件路径三层齐备（第 2 章已安装完成）；无 hooks 的平台（Zed、Antigravity IDE）只有第一层，需要手动放置教师文件（见 10.3 节）。

7.2 官方决策树

官方 `SKILL.md` 给出的判断流程可直接套用到任何一次操作前：

```
即将运行命令 / 读文件 / 调接口?
│
├── 命令在 Bash 白名单内（文件变更、git 写、导航、echo）？
│   └── 直接用 Bash
│
├── 输出可能很大，或拿不准?
│   └── 用 ctx_execute 或 ctx_execute_file
│
├── 抓网页文档或 HTML 页面?
│   └── 用 ctx_fetch_and_index，再用 ctx_search
│
├── 使用 Playwright（导航、快照、控制台、网络）？
│   └── 必用 filename 参数落盘，随后 ctx_index(path) 或 ctx_execute_file(path)
│
├── 处理其他 MCP 工具的输出（Context7、GitHub API 等）？
│   ├── 输出已在此前进入上下文 → 直接用，不要再索引
│   ├── 要反复查询 → 先存文件，再 ctx_index(path) → ctx_search
│   └── 一次性提取 → 先存文件，再 ctx_execute_file(path)
│
└── 读文件是为了分析或汇总（而非编辑）？
    └── 用 ctx_execute_file
```

其中 Playwright 分支属于一个自成体系的工作流（快照动辄 10 万至 13.5 万 token，第四章尺寸的输出对它只是零头），第 14 章单独展开。

7.3 情形选型总表

把高频工作的「情形 → 工具」映射整合为一张总表（整合官方 SKILL 表与第六章规则）：

情形	工具	典型示例
调用接口	`ctx_execute`	`fetch('http://localhost:3000/api/orders')`
数据型 CLI	`ctx_execute`	`gh pr list`、`aws s3 ls`、`kubectl get pods`
运行测试	`ctx_execute`	`npm test`、`pytest`、`go test ./...`
git 数据类操作	`ctx_execute`	`git log --oneline -50`、`git diff HEAD~5`
容器与集群检查	`ctx_execute`	`docker stats --no-stream`、`kubectl describe pod`
读日志或数据文件	`ctx_execute_file`	access.log、error.log、CSV、JSON、YAML
读源码做分析	`ctx_execute_file`	数函数、找模式、提指标
抓外部文档	`ctx_fetch_and_index`	React、Next.js、Zod 文档入索引
Playwright 快照（多问）	`filename` 落盘 → `ctx_index` → `ctx_search`	见 14.2 节
Playwright 快照（一次提取）	`filename` 落盘 → `ctx_execute_file`	见 14.3 节
清空知识库	`ctx_purge(confirm: true)`	永久删除全部已索引内容

7.4 自动触发场景清单

以下场景出现时，按官方路由规则应直接使用 context-mode，无须使用者点名：API 调试（「调这个接口」「看响应里哪出错了」）、日志分析（「看下日志」「哪些报错」）、测试运行、git 历史（最近提交、分支差异）、数据检视（「看下这份 CSV」「解析 JSON」）、基础设施（容器、Pod、S3、运行服务）、依赖审计（过期包、安全审计）、构建输出（编译错误、告警）、代码度量（行数、TODO、函数数）、文档查询（「查下文档」「API 参考」）。

7.5 子代理自动继承

通过 Task 等机制派生的子代理（subagent）无须手工重复路由要求：PreToolUse 钩子会把沙箱工具的路由指引自动注入子代理，使用者写自然语言任务描述即可。这一设计消除了「主代理记得、子代理忘」的常见缺口。

7.6 无 hooks 平台的自查

在没有 hooks（或 hooks 不完整）的平台上，7.1 节的强制层缺席，应主动做两件事：其一，放置对应的教师文件（放法见各平台安装小节，例如 Zed 的 `AGENTS.md`、Cursor 的 `context-mode.mdc`、Kiro 的 `KIRO.md`）；其二，在工作方式上保持「数据量大时点名工具」的习

惯，把 7.3 节的总表当作自查清单使用。按官方对照数据，有 hooks 与无 hooks 的执行率差别约为 98% 对 60%，后者的差距主要由漏网调用贡献。

第三篇 高级篇

> 本篇解剖四套机制：检索为什么快而准（第 8 章）、会话如何在压缩后失而复得（第 9 章）、路由如何做到程序化强制（第 10 章）、安全与配置如何自定义（第 11 至 12 章）。完成本篇后，读者应当能够根据机制解释观测到的行为——例如为什么某次检索的相关性偏低、为什么某平台的会话恢复不完整——并作出针对性的配置调整。

第 8 章 检索排名机制深入

第 6 章把 `ctx_search` 当作查询接口使用；要解释「为什么这两个词检索出的片段比那两个词准」「为什么拼写错了照样能命中」，需要进入内部先从入库端看起：索引与检索采用 SQLite 的 FTS5 全文检索扩展，在本文语境中，它是一个把文本切成可检索单元并倒排存储的虚拟表；排序采用 BM25 概率相关性算法，依据词频、逆文档频率与文档长度归一化对片段打分（首次概念均为铺垫，正文逐一展开）。本章按「入库 → 双策略检索 → 三层后处理」的顺序解剖整条链路。

8.1 入库端：分块、词干化与标题权重

`ctx_index` 把 markdown 按标题切块并保持代码块完整，随后写入 FTS5 虚拟表；入库端有三处细节决定检索质量的上限。

其一，分块索引时应用了 Porter 词干化（Porter stemming，一种把单词还原为词干的标准化算法）：`running`、`runs`、`ran` 在索引端折叠为同一词干，因此查询用词不必与原文逐字一致。其二，标题与各级标题在 BM25 打分中占五倍权重，使「按标题找章节」这类导航型查询的召回明显更准。

底层 SQLite 实现按运行环境自动选择：Bun 上使用 `bun:sqlite`，Node.js >= 22.5 上使用内置的 `node:sqlite` 模块，其余环境依赖 `better-sqlite3`；这一选择对使用者透明。

8.2 检索端：双策略并行与 RRF 融合

每次 `ctx_search` 会并行跑两个检索策略，再用倒数排名融合（Reciprocal Rank Fusion，简称 RRF；一种把多个排序列表合并为一个结果的算法，思路是排名靠前的结果按比例计分、两路都靠前的结果浮到最上面）合并：

- **Porter 词干化匹配策略**：在 FTS5 的 MATCH 查询中启用 porter 分词器，负责「词形不同、词义相同」的召回，例如 `caching` 能同时命中 `cached`、`caches`、`cach`。
- **三元子串策略（trigram）**：把文本切成三字符滑窗建立索引，负责「精确子串」的召回，例如查 `useEff` 能命中 `useEffect`，查 `authenticat` 能命中 `authentication`。

RRF 融合替代了早期版本「porter 无结果才尝试 trigram」的级联回退设计，现在两路始终并行、任意一路命中的内容都进入候选集。对使用者的含义直接可查：检索标识符、代码符号等短片段时，子串策略保证不漏；检索语义词时，词干策略保证不因词形差异遗漏。

8.3 邻近重排

多词查询在 RRF 之后还有一层重排：查询词在片段中彼此靠近的结果获得加权。典型例子是检索「session continuity」时，两个词紧挨着出现的段落，会排在「session」与「continuity」各占一处、相距数段的结果之前。查询词越完整地以相邻形态出现在某片段，该片段越应被视为「讲的就是这件事」。

8.4 模糊纠错

检索词进入查询前会先经 Levenshtein 编辑距离（衡量两个字符串相差最少需要多少次增、删、改一个字符）纠正拼写：`kuberntes` 被纠正为 `kubernetes`，`autentication` 被纠正为 `authentication`，纠正后再重新检索。这意味着模型输出的查询词带拼写噪音时，检索质量受影响有限。

8.5 智能片段

检索结果的展示不采用「按从头截取前 N 字符」的传统截断，而是定位查询词在内容中的命中位置，围绕命中位置截取窗口返回。当命中出现在文档中后段时，传统截断会错失全部相关内容，窗口式提取则始终围绕命中点。

8.6 调优实践汇总

综合 8.1 至 8.5 节的机制，日常检索可以归纳为四条调优动作：

现象	机制依据	调优动作
命中一堆无关长文	BM25 的 OR 语义按命中词数排序，常见词每处都命中	加上 2 至 4 个技术性名词；或用 `source` 限定来源
想找的词只是部分匹配	子串策略按三字符滑窗召回	直接检索标识符片段（如 `useEff`）即可命中
报错词拼写存疑	Levenshtein 纠错自动处理	不必先花轮次确认拼写，直接检索
需要多个问题	渐进节流惩罚连续调用	一个 `queries` 数组一次问完

第 9 章 会话延续与压缩恢复

上一章结束在检索机制；本章转到事件机制——上下文被压缩前后，context-mode 如何让工作状态不丢。9.1 节给出六类钩子在 Claude Code 中的分工，9.2 节列出被捕获的事件类型，9.3 节按时间顺序走完「压缩 → 快照 → 恢复」全流程，9.4 至 9.6 节补齐触发方式、平台差异与「新会话即清除」的默认语义。

9.1 六类钩子的分工

以主线平台 Claude Code 为例，会话延续由六类钩子协作完成（各平台达成度不同，见 9.5 节）：

钩子	职责	关键输出
PreToolUse	执行前路由强制	拦截或改道到大输出工具；同时记录被使用者否决的调用（拒绝方案）
PostToolUse	事件捕获	把每类有意义的工具活动抽成结构化事件落库
UserPromptSubmit	请求捕获	记录使用者请求、裁决、偏好与堵点描述
Stop	回合收尾	记录模型回合结束时的状态；Cursor 的 stop 钩子还可追加后续消息以延续循环
PreCompact	压缩前快照	生成分优先级的快照存入 `session_resume` 表
SessionStart	压缩后恢复	取回快照、重建工作状态并注入路由指引

9.2 捕获哪些事件

被捕获的事件共 23 类，按 P1（关键）至 P4（低）四档优先级分层；分层决定内容在压缩时的取舍顺序（优先级低的先被裁掉）。

P1（关键）：文件操作（读取、编辑、写入、glob、grep）；任务（创建、更新、完成）；计划（进入、退出、批准、拒绝、写盘）；项目规则文件（`CLAUDE.md`、`GEMINI.md`、`AGENTS.md` 的路径与内容，会话开始时捕获）；使用者的每一条消息（用于恢复「最后一次请求」）。

P2（高）：使用者裁决与偏好（「改用 X」「不要 Y」）；git 操作（checkout、commit、merge、rebase、stash、push、pull、diff、status）；错误与修复配对（错误发生又在稍后被解决）；发现的约束（「不支持」「无权限」）；堵点（「卡在 X」「等待 Y」，直至消除）；被拒绝的调用；环境变更（工作目录切换、venv、nvm、conda、工作树、包安装）；完成态子代理的结论（截取前 500 字符）；同一工具连续三次以上相似输入的循环征兆。

P3（普通）：超过 5 秒的慢调用（工具名与耗时毫秒数）；全部 `mcp__` 工具的调用计数；子代理的发起与结束；斜杠命令调用；引用的外部资源（URL、issue 号，自动去重）。

P4（低）：会话模式分类（调研、实现、评审）；使用者的角色性指示（「按资深工程师的视角处理」）；超过 1 KB 的大段粘贴数据的引用记录。

9.3 压缩与恢复的完整时序

按时间顺序，一次上下文压缩要发生以下动作：

1. **压缩前（PreCompact）**：从 SQLite 读出该会话的全部事件，组装成分优先级的 XML 快照，预算不超过 2 KB；预算吃紧时按「P4 → P3 → P2」顺序先裁低优先级，而关键状态（活动文件、任务、规则、裁决）始终保留。快照写入 `session_resume` 表。
2. **压缩后（SessionStart，来源标记为 compact）**：取回快照；把结构化事件写成文件并自动索引进 FTS5（细节可用 `ctx_search` 按需捞回）；生成一份 19 个栏目的会话指南（Session Guide）并注入 `<session_knowledge>` 指令。
3. **模型视角**：拿到指南中「最后一次请求」「任务清单（含完成状态）」「关键裁决」「已改文件」「未解决错误」等栏目，直接从上次的提示继续，无须再向使用者确认当前进度。

会话指南的 19 栏为：最后一次请求、任务、计划、关键裁决、已改文件、未解决错误、约束、堵点、git 操作、项目规则、用过的 MCP 工具、子代理任务、用过的斜杠命令、被拒绝的方案、外部引用、环境、大段数据引用、会话意图、使用者角色。

9.4 恢复的三种触发方式

- `--continue`、`--resume` 或 `/resume` 开启会话：读取上一会话快照，恢复工作状态。
- `/resume` 从历史列表挑选较早（非最近一次）的旧会话：编号对应的实况事件表为空时，回退读取该项目最近一份未消费的快照再行恢复；由使用者挑对话、由 context-mode 补状态。
- 自然压缩触发的恢复：如 9.3 节流程，自动执行，不必人工干预。

9.5 平台达成度

会话延续包含「捕获、快照、恢复」三个环节，各平台达成度如下表（评级为官方口径）。

等级	平台	说明
完整	Claude Code、Gemini CLI、VS Code Copilot、JetBrains Copilot、OpenCode、KiloCode	三环节齐备
高	GitHub Copilot CLI、OpenClaw、Pi Coding Agent、OMP（Oh My Pi）	关键环节齐备，个别捕获类目有出入
部分	Cursor、Codex CLI、Antigravity CLI（`agy`）、Kiro	通常缺恢复或个别钩子（如 Cursor 的 `sessionStart` 被校验器拒绝）
无	Antigravity IDE、Zed	无钩子体系，不做事件捕获

两类常见差异提醒：Gemini CLI、VS Code Copilot 与 JetBrains Copilot 均无 UserPromptSubmit 等价钩子，因此「使用者裁决与偏好」不被捕获（文件、任务、git、错误仍然完整）；Codex CLI 的 PreToolUse 钩子支撑的动作有限，但 MCP 工具不受影响。

9.6 新会话即清除

会话数据的默认生命周期跟随使用者的选择：以 `--continue`、`--resume` 或 `/resume` 续接，上一会话数据被取用；若开启全新会话且没有续接动作，上一会话的事件与快照立即删除。这套默认带来两个直接推论：想保留可恢复性，续接是必须的显式动作；换项目或换任务时，系统不会自动携带旧状态进入新会话，等于「新会话即净空」。知识库内容是例外，其清理由 14 天自动清理与 `ctx_purge` 管理（见 6.3 至 6.4 节）。

第 10 章 路由强制执行与平台兼容

第 7 章介绍了三层引导；本章从实现角度补充第二、第三层的平台差异：哪些平台有哪类钩子、没有钩子的平台如何补偿（10.1 至 10.2 节），各平台在协议上的关键断点（10.3 节），以及多平台同机安装时的识别与隔离（10.4 节）。

10.1 强制与教学的边界

hooks 与教师文件的能力差异值得以表格精确表达：

途径	位置	能力	官方口径命中率
hooks（钩子）	宿主进程触发	在工具执行前程序化拦截、阻断、改写入参；执行后捕获事件	约 98% 按路由执行
教师文件	项目目录或宿主全局目录	提示词引导，不能阻断任何调用	约 60% 按路由执行

教师文件未能覆盖的那约 40%，代价可以量化：无钩子状态下，一次未引导的 `curl` 或 Playwright 快照即可带入约 56 KB，抵销一整个会话的节省。官方建议明确：在支持 hooks 的平台上始终启用 hooks。

历史上 context-mode 曾在会话启动时自动把教师文件写入项目目录，该行为已被移除以避免污染 git 工作树（Issue #158、#164）；目前除无钩子平台外，不再落盘任何文件。

10.2 平台能力矩阵

下表按官方口径给出三类核心钩子与 MCP 工具在各平台的分布（「插件」指以平台插件的进程内函数形式实现；「(ext)」指以扩展事件形式实现；「部分」指能力受限，细节见 10.3 节）。

平台	MCP 工具	PreToolUse	PostToolUse	SessionStart	PreCompact	会话延续评级
Claude Code	是	是	是	是	是	完整
Qwen Code	是	是	是	是	是	官方未单列评级
Gemini CLI	是	--	是（AfterTool）	是	是（PreCompress）	高
VS Code Copilot	是	--	是	是	是	高
JetBrains Copilot	是	--	是	是	是	高
GitHub Copilot CLI	是	是	是	是	是	高
Cursor	是	是	是	--（校验器拒绝）	--	部分
OpenCode	原生插件	插件	插件	经 `experimental.chat.system.transform` 代理	插件	完整
KiloCode	原生插件	插件	插件	同 OpenCode 代理方式	插件	完整
OpenClaw 网关	是	插件	插件	插件	插件	高
Codex CLI	是	是（仅 deny）	是	是	是（依版本触发）	部分
Kimi Code	是	是	是	是	是	官方未单列评级
Antigravity IDE	是	--	--	--	--	无
Antigravity CLI（`agy`）	是	有界	是（仅捕获）	--	--	部分
Kiro	是	是	是	--（`agentSpawn` 未接）	--	部分
Zed	是	--	--	--	--	无
Pi Coding Agent	是	是（ext）	是（ext）	是（ext）	是（ext）	高
OMP（Oh My Pi）	是	插件	插件	插件	插件	高

10.3 各平台的关键差异注记

以下注记覆盖在选型或排障时最可能踩到的差异点：

- **GitHub Copilot CLI**：hooks 配置键为驼峰式（`preToolUse`、`postToolUse`、`preCompact`、`sessionStart`、`userPromptSubmitted`、`agentStop`），响应字段位于顶层，与其他平台不同；插件路径会一并注册 MCP、hooks 与路由技能。
- **OpenCode 与 KiloCode**：无真正的 SessionStart 钩子，以 `experimental.chat.system.transform` 作为代理在会话启动时注入路由与恢复状态，以 `chat.message` 捕获使用者消息（Issue #14808、#5409）；两者配置形态一致，仅文件名不同（`opencode.json` 与 `kilo.json`）。

- **Cursor**：`sessionStart` 已在其文档中列出但被其校验器拒绝（社区报告见 Cursor 论坛），压缩后的状态恢复暂不可用；此外 hook 响应中的 `additional_context` 能被接受但不透出给模型（论坛帖 #155689），路由感知依赖 `.mdc` 规则文件。
- **Codex CLI**：PreToolUse 目前只支持 deny 类动作，入参改写待上游支持 `updatedInput`（openai/codex#18491）；PreToolUse 上下文注入不支持，改经 PostToolUse 与 SessionStart 完成；PreCompact 是否触发依 Codex 版本而定。
- **Kiro**：`agentSpawn`（SessionStart 等价物）与 `stop` 未接线，恢复不可用；路由由 `preToolUse` 与 `postToolUse` 承担强制与捕获，教师文件 `KIRO.md` 补初始感知。
- **OpenClaw**：以原生网关插件运行，钩子经 `api.on()` 注册，要求网关版本晚于 2026.1.29（PR #9761）；旧版本上生命周期钩子静默失效，适配器转为从数据库重建快照（精度下降但关键状态保留）。
- **OpenClaw 之外的「无钩子双件套」**：Antigravity IDE 与 Zed 只有 MCP 一层，路由感知完全依赖手动复制教师文件；两者的识别走 MCP 握手，无须额外配置。
- **Pi 与 OMP**：均以扩展或插件注册 `tool_call`、`tool_result`、`session_start`、`session_before_compact` 四个事件，达成与其他平台钩子等价的路径；Pi 的工具输出提供折叠/展开键（默认 `Ctrl+O`），长输出在界面层面也可压缩显示。

10.4 多平台同机安装时的识别与隔离

同一台机器并装多个平台时，context-mode 需要判断「当前会话运行在哪个平台」，否则可能误接配置。项目为此使用两类机制（均已核实）：

- **自动识别**：多数场景通过 MCP 握手中的客户端标识（`clientInfo.name`）或平台特征（如 `~/.omp/` 目录、`PI_CODING_AGENT_DIR` 环境变量）判断；GitHub Copilot CLI 另有一份由 context-mode 写入的标记文件作为裸 shell 环境下的判据。
- **平台钉扎**：Copilot CLI 与 `agy` 的插件安装时会把 `CONTEXT_MODE_PLATFORM` 环境变量固定写入配置（分别为 `copilot-cli` 与 `antigravity-cli`），避免与同机已装的 Claude Code（默认会读取 `~/.claude/`）相互覆盖身份（Issue #774）。

存储隔离方面，OMP 的数据根放在 `~/.omp/context-mode/`，与 Pi 安装互不共享会话库、内容索引与统计文件（Issue #473）；其余平台沿用各自的默认根或 `CONTEXT_MODE_DIR` 指定位。

第 11 章 安全模型

工具收窄了上下文，但没有收窄权限边界；正因如此，安全设计的核心问题是「沙箱会不会成为绕过权力的暗门」。context-mode 的答案是把宿主已有的权限规则原样延伸到沙箱内（11.1 节），再辅以文件路径边界（11.2 节）、网络目标加固（11.3 节）与落库脱敏（11.4 节）；11.5 节说明私有边界——哪些数据始终不出本机。

11.1 权限规则的原样复用

context-mode 直接读取 Claude Code 格式的 `permissions` 设置（项目根 `.claude/settings.json` 或全局 `~/.claude/settings.json`），并把同样规则套用到 `ctx_execute`、`ctx_execute_file`、`ctx_batch_execute` 之内。规则语义与 Claude Code 一致：

- 规则形如 `Tool(匹配式)`，`*` 表示任意；`deny` 一旦命中优先于 `allow`；项目级更具体的规则覆盖全局规则。
- 链式命令按 `&&`、`;`、`|` 拆开逐段检查：`echo hello && sudo rm -rf /tmp` 这类拼接命令会因 `sudo` 段命中 `Bash(sudo *)` 而被拒。
- 未配置任何规则的部署不受影响；需要收紧时按需补规则，例如：

```
{
  "permissions": {
    "deny": [
      "Bash(sudo *)",
      "Bash(rm -rf /*)",
      "Read(.env)",
      "Read(**/.env*)"
    ],
    "allow": [
      "Bash(git:*)",
      "Bash(npm:*)"
    ]
  }
}
```

除 Codex CLI 需要其 hooks 文件就位外，其余平台一律读取同一套 Claude Code 格式。

11.2 文件路径边界：ctx_execute_file 限制在项目内

`ctx_execute_file` 的 `path` 解析出项目根之外的目标（绝对路径、`../../` 上溯、或指向项目外的软链接）时，调用被拒并返回 `File access blocked`。该防线默认开启，对应一个已披露的真实逃逸案例（Issue #852）：某代理在宿主侧被沙箱拒绝读取项目外文件后，改道 MCP 沙箱重试，而宿主的 MCP 审批交互只展示工具名、不展示入参，使逃逸对审批者不可见。

确实需要处理项目外文件（例如 `/var/log` 下的共享日志）时，使用与宿主 Read 工具相同的 `permissions.allow` 规则放行，没有 context-mode 专属开关：

```
{
  "permissions": {
    "allow": ["Read(/var/log/**)"]
  }
}
```

值得记住的一条边界事实：`ctx_execute` 与 `ctx_batch_execute` 会运行任意代码并继承进程的文件系统访问权，路径护栏只是文件读取工具上的纵深防御层；对两类执行工具的任何一次审批，都应视为在批准任意代码执行，宿主级沙箱不应因此关闭。官方也把工具审批标题改为「Run code in a sandbox…」一类的直述文案，帮助不熟悉的评审者判断动作类别。

11.3 网络抓取目标加固

`ctx_fetch_and_index` 对抓取目标执行默认白名单策略：

- 协议仅允许 `http:` 与 `https:`（`file:`、`gopher:`、`javascript:`、`data:` 一律拒绝）。
- 云元数据端点与链路本地地址（`169.254.0.0/16`，含 AWS、GCP、Azure 通用元数据地址 `169.254.169.254`）硬封锁，即使主机名解析到该地址也被拦（防 DNS 重绑定）；多播与保留段（`224.0.0.0/4`、`0.0.0.0/8`、IPv6 `ff00::/8`、`fe80::/10`）同样封锁。
- 回环与内网段（`localhost`、`127.x`、`10.x`、`172.16-31.x`、`192.168.x`、IPv6 `::1`、`fc00::/7`）默认放行，使本地开发服务器与内网抓取可用。

在共享或 CI 环境中部署 context-mode 时，建议收紧内网放行，设置环境变量 `CTX_FETCH_STRICT=1`：除上述硬封锁段外，连回环与内网段一并禁止。本地开发场景不应默认开启该变量，其代价是 `localhost` 与内网地址的抓取也会一并被封锁。

11.4 落库脱敏

所有 `mcp__` 工具调用在事件入库前，其 `tool_input` 先按键名匹配掩码化：`authorization`、`auth_token`、`access_token`、`refresh_token`、`bearer`、`token`、`secret`、`password`、`passwd`、`pwd`、`api_key` / `apikey` / `x_api_key`、`cookie` / `set-cookie`、`signature`、`private_key`、`client_secret`（大小写不敏感，连字符与下划线亦不敏感）被替换为 `[REDACTED]`。实现在 PostToolUse 钩子的掩码逻辑中，目标是让 MCP 参数里的凭据不落 SQLite。

11.5 隐私架构

数据不出本机是项目的明确设计决定：无遥测、无云同步、无用量上报、无需账号；代码、提示、会话数据全部本地，SQLite 库存放在使用者的主目录。唯一涉及浏览器的功能是 `ctx_insight` 打开托管于 context-mode.com/insight 的机构分析页面，属于按需开启的团队分析，不改沙箱数据面。许可层面的 ELv2 限制（不得作为托管服务出售、不得移除许可声明）也已在 1.2 节说明。

第 12 章 配置、环境变量与升级

前三章的机制都有配置入口；本章把可调项集中列出（12.1 至 12.2 节），说明版本升级的路径与已知坑（12.3 至 12.4 节），最后给出各平台的面卸载做法（12.5 节）。

12.1 存储布局与 CONTEXT_MODE_DIR

存储根目录确定两块数据的落位：会话与统计在 `<根>/sessions`，知识库索引在 `<根>/content`。根目录本身由 `CONTEXT_MODE_DIR` 决定，未设置时取各适配器默认（例如 Claude Code 为 `~/.claude/context-mode`，Codex CLI 为 `~/.codex/context-mode`）。该变量自 v1.0.147 起提供，约束有三：设置为空或空白视同未设置（`ctx_doctor` 会明示）；非空值必须是绝对路径；`~` 不做展开。

12.2 环境变量一览

context-mode 当前公开使用的三个环境变量如下（2026-10-30 依照官方 README 核实）：

变量	默认	用途
`CONTEXT_MODE_DIR`	适配器默认根	存储根目录；会话与统计在 `<根>/sessions`，索引内容在 `<根>/content`（v1.0.147+）
`CTX_FETCH_STRICT`	未设置	设为 `1` 时在默认网络封堵之上再禁回环与内网段（`127.x`、`10.x`、`172.16-31.x`、`192.168.x`、IPv6 `::1`、`fc00::/7`），用于共享或 CI 环境
`CONTEXT_MODE_EXTERNAL_MCP_NUDGE_EVERY`	`10`	外部 MCP 调用需包裹进 `ctx_execute` 的提示重发节奏：每第 N 次匹配调用重提一次；取值范围 1 至 100，非法值回退为 10；设为 `1` 最激进、每次调用附加约 250 token 的提示

第三个变量的出处值得说明一下：早期实现（PR #529）每会话只提醒一次，在 MCP 重型会话里，提醒一被上下文压缩挤掉就再也不会出现（Issue #567 的 Jira、Slack、Notion 高频场景），现行默认每 10 次重提保持其在近期窗口内。重度使用 MCP 的项目可把该值调小，代价按需承受。

12.3 升级路径

升级有三条等效路径：会话内 `/context-mode:ctx-upgrade`（或直接输入 `ctx upgrade`）；终端执行 `context-mode upgrade`；手动重装 `npm install -g context-mode@latest`。ctx_upgrade（会话内升级工具）的动作包括从 GitHub 拉取最新版本、重建制品、迁移缓存并修复 hooks，完成后再跑一次 `ctx_doctor` 验证。

12.4 已知升级坑

以下问题在升级或迁移时最常出现（均有官方出处）：

- **OpenCode 与 KiloCode 的双注册**：配置里同时存在 `plugin: ["context-mode"]` 与旧版 `mcp.context-mode` 时，`ctx_` 工具会一个都不注册——插件路径本会正确抑制重复的 MCP 条目，但旧 MCP 键本身会让加载器混淆。`context-mode upgrade` 会自动移除旧 MCP 条目并保留其他 MCP 服务器；v1.0.140+ 在故障发生时以 stderr 提示并给出相同指引。
- **全局可执行文件旧于 hooks**：某些平台的钩子命令直接执行全局 `context-mode` 可执行文件（如 Copilot CLI 的 `context-mode hook copilot-cli pretooluse`），若全局版本仍未支持某平台，钩子会静默失效（不阻塞工具调用，但路由与捕获不再发生），需要 `npm install -g context-mode@latest` 升级全局可执行文件。
- **Codex CLI 的 PreCompact 分版触发**：Codex CLI 0.130.0 才出现该事件；更早构建的版本不会生成压缩前快照（不会干预压缩后恢复，只会没有快照可取）。
- **Node 可见性**：以插件形式安装的 Codex 等，仍需要宿主进程能找到 `node`；插件不会自带 Node，也不自动继承登录 Shell 的 PATH 修复。

12.5 卸载

卸载涉及三层，最好按序处理，确保数据也一并处理到本地清理的范围：

1. **平台注册面**：Claude Code 在插件管理（`/plugin`）中卸载插件并移除 marketplace 注册；其他平台按安装步骤逆操作：删除各自配置文件中的 MCP 与 hooks 条目（如 `opencode.json` 的 `plugin` 键、`.github/hooks/context-mode.json`、`.kiro/hooks/context-mode.json`、`agy plugin uninstall context-mode` 等）、可移除教师文件。
2. **全局安装面**：`npm uninstall -g context-mode`（或 Bun 等价命令）移除可执行文件。
3. **数据面**：先在应用内执行 `ctx_purge(confirm: true)` 清空知识库；需要彻底清理存储时，删除存储根目录（`CONTEXT_MODE_DIR` 指定处或各平台默认根）下的 `sessions` 与 `content` 目录。卸载平台配置不会自动清理这些本地数据。

第四篇 精通篇

> 本篇完成从「会用工具」到「稳定产出」的转轨：Think in Code 的方法论与模式库（第 13 章）、浏览器与外部 MCP 的完整数据流（第 14 章）、CLI 运维与多平台部署（第 15 章）、综合实战（第 16 章）、以及基准解读、故障排除与团队推广（第 17 章）。完成本篇后，读者应能在个人与团队两个层面把 context-mode 纳入日常工作与部署流程。

第 13 章 Think in Code 方法论与模式库

读到本篇，读者已经积累了十几组命令的用法；本章要把散点收拢为一种工作方式：把模型当作代码生成器，把算力消耗大的处理交由脚本执行。README 将其表述为「把 LLM 从数据处理者变为代码生成者」；一次会话量级的差异可以大到 47 次 Read 调用共约 700 KB 的原始数据，对照一次 `ctx_execute` 约 3.6 KB 的结论输出。

13.1 一个对照例子

以下官方示例统计 `src` 目录下每个 TypeScript 文件的行数，一次调用完成遍历、汇总与输出：

```
// 对照：47 次逐文件 Read 约计 700 KB；本调用约 3.6 KB
ctx_execute("javascript", `
  const files = fs.readdirSync('src').filter(f => f.endsWith('.ts'));
  files.forEach(f => console.log(f + ': ' + fs.readFileSync('src/'+f,'utf8').split('\n').length + ' lines'));
`);
```

该例不是特例，而是一个可迁移的模板：凡是「读很多、读出来只是为了算一个数」的任务——数函数、查 TODO、统计依赖版本——都套用「先遍历、后聚合、最后打印汇总」的写法。

13.2 输出纪律

官方路由规则为沙箱代码归纳出全套输出纪律，逐条对应到机制含义：

规则	机制含义
必须有打印，`stdout` 是唯一回程	不打印等于调用白费（见 4.1 节）
打印分析结论，不打印数据转储	避免「换个名目的洪水」（4.1 节第二条纪律）
打印要具体：ID、行号、精确值	片段式输出要经得起后续追问
需要编辑的文件继续用宿主的 Read 工具	context-mode 定位于分析，不是编辑流程的替代
Bash 白名单命令才直走 Bash	见 4.6 节白名单表
大数据禁止 `ctx_index(content: ...)`	6.3 节红线：内容会让上下文先载入一次工具入参
Playwright 抓取必带 `filename` 参数	详见第 14 章
已在上下文中的数据不重复索引	等于把同一份数据的成本再付一次，直接使用或先落盘再走 `path`

13.3 JavaScript/TypeScript 模式

两类高频场景可直接复用：

- **接口调试**：4.3 节的订单校验示例覆盖「拉取—校验—打印异常清单」的最小闭环。
- **代码库统计**：13.1 节的行数统计模板可换体裁复用，例如把 `.ts` 换为其他扩展名，或把行数统一指标（TODO 数量、函数数量等，属 7.4 节自动触发场景中的代码度量一类）。

13.4 Python 模式

`ctx_execute_file` 的 `FILE_CONTENT` 预载模式（5.2 节）覆盖日常最重要的一类任务：把一份大文件交给一段短脚本。与 `json.loads(FILE_CONTENT)` 同构的还有 csv、YAML 等标准库写法；流程一律是「加载后聚合，打印结论」。

13.5 Shell 模式

Shell 沙箱适合「工具链已在 PATH 里」的场景：管道内的 `grep`、`awk`、`jq` 就地过滤，最终把结果行打印出来。4.3 节的 `gh pr list --json` 示例演示了「用 CLI 自己的查询参数先收窄、再进上下文」的思路。官方反模式特别警告相反的写法：以「先取全量再 `| head -20` 截断」浏览结果会把未查看的数据直接丢掉、无法回补，正确做法是在沙箱内分析全量数据后打印结论（见 13.6 节条目）。

13.6 反模式清单

来自官方 `SKILL.md` 反模式章节，是 13.2 节纪律的数量化版本，按「错误做法—后果—正确做法」整理：

错误做法	后果	正确做法
Bash 里 `curl http://api/...`	约 50 KB 响应整个进入上下文	`ctx_execute` 中以 fetch 调用
Bash 里 `cat large-file.json`	整份文件进入上下文	`ctx_execute_file` 处理
Bash 里 `gh pr list` 交给原始 JSON	原始 JSON 全量进入上下文	沙箱内走 `--jq` 等参数先收窄
用 `| head -20` 截断浏览	未查看的数据直接丢失，无法回补	沙箱分析全量数据，打印结论
Bash 里 `npm test`	完整测试输出进入上下文	`ctx_execute` 捕获并出汇总
`browser_snapshot()` 未带 `filename`	约 13.5 万 token 的快照进入上下文	`browser_snapshot(filename: "/tmp/snap.md")`
控制台与网络抓取未带 `filename`	全部输出进入上下文	同上，落盘后走文件路径
把大数据传给 `ctx_index(content:)`	数据先作为工具参数进入上下文一次	换 `ctx_index(path: ...)` 服务端读磁盘
把 MCP 响应再传给 `ctx_index(content:)`	响应已在上下文，形如同一份数据付两次费	直接使用；或先落盘，再按 `path` 走
依赖 `browser_navigate` 自带的快照	导航响应附带整页快照，等于未受控的大输出	检查性的抓取一律单独发起带 `filename` 的 `browser_snapshot`
指望 `ctx_stats` 清空或清理什么	`ctx_stats` 是只读统计；被误以为删除时会产生误解	删除用 `ctx_purge(confirm: true)`（6.4 节）

第 14 章 浏览器与外部 MCP 集成工作流

第 7 章决策树预留了 Playwright 分支；本章把它展开为三套完整工作流。其背景是一组数字：一次 Playwright 的无障碍树快照约 1 万至 13.5 万 token；除此之外，任何返回大体积外部 MCP 数据的场景共享同一套「落盘 → 服务端读取」的处置思路。

14.1 四条路径的成本对照

官方给出四种处置方式的上下文成本对照，可当作本节的地图：

数据流路径	上下文成本	判定
`browser_snapshot()` 直接取原始输出	约 13.5 万 token	错误做法
快照原始值再传给 `ctx_index(content:)`	约 27 万 token（二次入上下文）	错误做法
快照带 `filename` 落盘 → `ctx_index(path)` → `ctx_search`	合计约 430 B	正确做法
快照带 `filename` 落盘 → `ctx_execute_file(path)`	合计约 250 B	正确做法

一句话概括为：数据流应当是「Playwright → 文件 → 服务端读取 → 上下文」，永远不会是「Playwright → 上下文 → 作为工具参数再上传 → 再回到上下文」。

14.2 工作流 A：快照 → 文件 → 索引 → 检索（多次提问）

同一份页面快照要反复提问时（表单结构、按钮清单、链接清单各查几轮），走索引路径：

```
第 1 步 browser_snapshot(filename: "/tmp/playwright-snapshot.md")
        → 只回执约 50 B 的落盘确认，而不是 13.5 万 token

第 2 步 ctx_index(path: "/tmp/playwright-snapshot.md", source: "Playwright snapshot")
        → 服务端读磁盘入库，回执约 80 B

第 3 步 ctx_search(queries: ["login form email password"], source: "Playwright")
        → 只回执命中的片段（约 300 B 起）
```

合计上下文约 430 B，对照的原始路径在 13.5 万 token 起步；这也是第 6 章 source 参数的实战形态：此处检索用 `source: "Playwright"` 即可命中名为 `Playwright snapshot` 的来源（部分匹配）。

14.3 工作流 B：快照 → 文件 → 沙箱（一次提取）

只做一次提取（拼一份页面元素清单）时，不必建索引，直接把文件交给沙箱：

```
// ctx_execute_file 内，FILE_CONTENT 即快照文件
const links = [...FILE_CONTENT.matchAll(/- link \"([^\"]+)\"/g)].map(m => m[1]);
const buttons = [...FILE_CONTENT.matchAll(/- button \"([^\"]+)\"/g)].map(m => m[1]);
const inputs = [...FILE_CONTENT.matchAll(/- textbox|- checkbox|- radio/g)];
console.log('Links:', links.length, '| Buttons:', buttons.length, '| Inputs:', inputs.length);
console.log('Navigation:', links.slice(0, 10).join(', '));
```

合计上下文约 250 B；出现在上下文里的只有这行汇总与首十个导航项。

14.4 工作流 C：控制台与网络输出

页面之外的控制台与网络信息同样照该套路走：

```
browser_console_messages(level: "error", filename: "/tmp/console.md")
→ 随后 ctx_execute_file 一次性提取，或 ctx_index(path) 建索引自查

browser_network_requests(includeStatic: false, filename: "/tmp/network.md")
→ 同上
```

14.5 两个已知的浏览器侧陷阱

- **导航自带快照**：`browser_navigate` 的返回里已经附了一整页快照；以它为检查依据等于走「未受控」的默认路径。官方建议：任何检查性需要都单独发起带 `filename` 的 `browser_snapshot`。
- **单实例限制**：Playwright MCP 使用单一浏览器实例，不是并行安全的设计；需要并行做多个页面的代理任务时，官方建议改走会话隔离的浏览器自动化（如 agent-browser），在宿主的执行工具中以独立子进程方式运行，再按本节工作流走文件路径处理输出。

14.6 外部 MCP 工具输出的三种处置

不仅 Playwright，任何会产生大体积数据的 MCP 工具（文档检索、GitHub 拉取、云清单）都适用同一处置决策：

1. **数据已在此前进入上下文**——直接使用，不再索引；重复支付的钱追不回。
2. **需要反复检索**——在沙箱或宿主端先把响应落盘，再 `ctx_index(path: ...)` 建库，进入 `ctx_search` 多轮问答。
3. **一次性提取**——落盘后 `ctx_execute_file` 在沙箱内完成提取。

12.2 节介绍的 `CONTEXT_MODE_EXTERNAL_MCP_NUDGE_EVERY` 变量控制的正是这一层的自动提醒频率：当模型反复把外部 MCP 调用暴露给上下文时，钩子会按设定的调用间隔重发「请包裹进 `ctx_execute`」的提示。

第 15 章 CLI 运维与新平台部署

到本章为止的操作都在 AI 会话内；本章转移到终端侧与部署侧：不进入会话也能完成的索引、检索、诊断与升级（15.1 至 15.3 节），从个人到团队的平台部署策略（15.4 节），以及用量度量与展示（15.5 节）。

15.1 终端命令全集

终端侧以下命令均可直接运行，无需 AI 会话：

```
context-mode doctor
context-mode index . --source project:my-app
context-mode search "authentication middleware" --source project:my-app
context-mode upgrade
context-mode insight          # 在浏览器打开托管的分析页面
bash scripts/ctx-debug.sh     # 生成一份可粘贴的完整诊断报告
```

逐条解释：`doctor` 等价于会话内 `ctx_doctor`，校验运行时、hooks、FTS5 与版本；`index` 把当前目录的内容入库，`--source project:my-app` 为入库内容取一个来源名（之后检索用同名过滤）；`search` 直接检索既有索引，返回与 `ctx_search` 相同规则的结果；`upgrade` 的动作等同 12.3 节；`insight` 打开 11.5 节提到的托管分析页面（与本地隐私边界的关系见该节说明）。

15.2 诊断脚本输出的信息构成

为排查而生的 `ctx-debug.sh` 会把以下项目汇成一份 markdown 报告：操作系统信息、各运行时版本、`better-sqlite3` 状态、平台适配识别结果、配置文件（脱敏后）、hooks 校验、FTS5 与 SQLite 自测、executor 自测、进程检查、会话数据库概览与环境变量。官方建议将其输出随 bug 报告一并粘贴。

15.3 会话内命令对照

所有终端命令都有会话内等价形态，转换规则是「输入短语音，模型改调 MCP 工具」：

想做的事	会话内输入	Claude Code 斜杠形式	背后工具
查看节省统计	`ctx stats`	`/context-mode:ctx-stats`	`ctx_stats`
诊断安装	`ctx doctor`	`/context-mode:ctx-doctor`	`ctx_doctor`
索引内容入库	`ctx index`	`/context-mode:ctx-index`	`ctx_index`
检索已索引内容	`ctx search`	`/context-mode:ctx-search`	`ctx_search`
升级	`ctx upgrade`	`/context-mode:ctx-upgrade`	`ctx_upgrade`
清空知识库	`ctx purge`	`/context-mode:ctx-purge`	`ctx_purge`（需 `confirm: true`）
打开分析页	`ctx insight`	`/context-mode:ctx-insight`	`ctx_insight`

15.4 多平台部署策略

结合第二、三篇的平台与机制资料，部署可以按规模分三类：

- **个人使用**：主力平台走插件完整路径（获得强制层与完整会话延续）；还想在其他平台试用的，先以 MCP-only 方式接入（2.5 节），再按第 10 章矩阵评估平台完成度。
- **多平台个人**：Claude Code 保持完整安装，其余平台按附录 C 逐个接入；注意 12.4 节的双注册等坑，调整后用 `ctx doctor` 逐平台验证。
- **团队**：以「有 hooks 的平台优先」为选型原则（10.1 节的 98% 与 60% 之差是真实的长期成本）；无钩子平台若纳入团队标配，教师文件由仓库统一维护，而不是靠每名成员手动复制，验收时以 `ctx doctor` 作为检查项；用量度量统一走 15.5 节的统计与（若采用）Insight 页面观测。

15.5 度量与展示

三层展示口径覆盖不同观察粒度：会话内 `ctx stats` 查当前会话与累计节省、各工具调用拆分与缓存指标（6.3 节）；Claude Code 状态栏持续显示金额换算与效率百分比（2.4 节）；`ctx_insight` 打开的托管页面面向团队整体视角观测。基准数字（17.1 节）只代表官方口径，部署后的判断一律以这三层本地实测为准。

第 16 章 综合实战案例集

各章的用法已经分开讲过；本章把它们按真实任务串成线。每个案例给出目标、可直接粘贴的提示词（保留官方英文原文）与预期路径，运行后在会话内输入 `ctx stats` 观察节省。

16.1 深度仓库调研

目标：对一个陌生仓库形成「架构、技术栈、主要贡献者、开放 issue、近期活跃」的立体认识，全程不把中间页装进上下文。

```
Research https://github.com/modelcontextprotocol/servers — architecture, tech stack,
top contributors, open issues, and recent activity. Then run /context-mode:ctx-stats.
```

预期路径：抓取与汇总在沙箱与知识库内完成，最终进入上下文的是各项结论；官方口径 5 次调用、62 KB（原始约 986 KB，节省约 94%）。这是 3.3 节场景的完整版，也是「批量 + 索引 + 检索」三者协同的范本。

16.2 git 历史分析

目标：把五百条提交沉淀为三张统计表。

```
Clone https://github.com/facebook/react and analyze the last 500 commits:
top contributors, commit frequency by month, and most changed files.
Then run /context-mode:ctx-stats.
```

预期路径：克隆与五百条提交的统计都在沙箱内执行；官方口径 1 次调用、约 5.6 KB。与「直接翻 log」的上下文成本对照已在 3.4 节给出。

16.3 日志与构建输出分析

目标：从 access.log 类高行数日志中抽出错误分布、高频路径与异常请求，不带走一个原始行。

要点回顾：日志经由 `ctx_execute_file`（或脚本内文件读取）进入沙箱，脚本做分组、计数与过滤，只打印错误段的结论；4.4 节的 intent 过滤会自动给超过 5 KB 的输出补一轮「只给意图相关段落」的降级。测试输出同理，13.5 节的 `npm test` 模板把「全量行」折成「退出码与失败清单」。

16.4 API 调试

目标：一次往返确认接口的问题所在（字段异常、负值、空值等），不在上下文中留原始响应。

统一按 13.3 节模式一的写法：拉取、逐类校验、打印异常清单，其中清单里的 ID 与具体值是后续修复的键。若接口量大（多端点并行），交给 5.1 节的 `ctx_batch_execute` 一次调用聚合。

16.5 本地服务与大体积 JSON

目标：复现「7.5 MB 响应、两万条记录、一条隐藏在 13000 号下标」的官方压力场景，观察沙箱把存储成本压到常数水平的能力。

```
Create a local server that returns a 7.5 MB JSON with 20,000 records and a secret
hidden at index 13000. Fetch the endpoint, find the hidden record, and show me
exactly what's in it. Then run /context-mode:ctx-stats.
```

预期路径：先由代理创建本地服务（此属写文件操作，走宿主普通工具），随后抓取与过滤在 `ctx_execute` 的沙箱内完成，进入上下文的只有那条隐藏记录。官方口径：原始 7.5 MB，上下文约 0.9 KB，节省约 99%。

16.6 文档检索工作流

目标：把某份官方文档入库后反复检索而不重复抓取。

```
Fetch the React useEffect docs, index them, and find the cleanup pattern
with code examples. Then run /context-mode:ctx-stats.
```

预期路径：`ctx_fetch_and_index` 抓取、转换、分块入库（TTL 缓存规则见 6.3 节），`ctx_search` 只回命中片段；官方口径 2 次调用、约 1.8 KB。注意 6.4 节的红线：文档入库走 `path` 或由抓取工具自动完成，不走 `content`。GitHub 托管的文档直接取 raw 直链（第 6 章末尾的写法）。

16.7 长任务会话演练

目标：验证 9.3 节的恢复时序在最真实的延迟形态下有效。

步骤：发起多步任务「创建带路由、测试与错误处理的 Express REST API」，推进使工具调用数超过 20 次，输入 `ctx stats` 看事件计数；随后让上下文自然触发压缩，恢复时序参见 9.3 节。预期结果是压缩后模型继续最后的提示而不重复询问任务内容，任务、文件与裁决俱在。前置于演练的确认事项同样来自第 9 章：以续接方式开启会话会带出上一会话存档，未续接的新会话则清空处理。

第 17 章 性能基准解读、故障排除与团队推广

本篇最后一章承担三件收尾工作：教读者正确解读官方基准（17.1 节），在遇到异常时按表自查（17.3 节），以及把个人经验升级为团队部署（17.4 节）。三个附表（附录 A 至 E）随后提供速查。

17.1 基准数据与方法论

官方 `BENCHMARK.md` 覆盖 21 个场景，README 摘录的八组代表数据如下（核实于 2026-10-30）：

场景	原始数据	进入上下文	节省
Playwright 页面快照	56.2 KB	299 B	99%
GitHub Issues（20 条）	58.9 KB	1.1 KB	98%
访问日志（500 请求）	45.1 KB	155 B	100%
Context7 React 文档	5.9 KB	261 B	96%
分析用 CSV（500 行）	85.5 KB	222 B	100%
git log（153 提交）	11.6 KB	107 B	99%
测试输出（30 套件）	6.0 KB	337 B	95%
仓库调研（子代理）	986 KB	62 KB	94%

两条解读要求：其一，这些数字是项目自测口径，读法是「量化上限参考」而非「收益承诺」；第三方评述同样提醒该数据来自项目自身的工作负载。收益与任务形态相关——测试、日志、快照类高频大输出的任务收益大，轻量交互类任务收益自然小。其二，在本机评估收益时以 `ctx_stats` 实测为准，观测方法按 15.5 节三层口径选择。整体会话量级的官方口径为：315 KB 压至 5.4 KB、会话时长从约 30 分钟延至约 3 小时。

17.2 成本构成与权衡预期

启用 context-mode 的「支出」主要有两类：hooks 与路由块的轻量提示注入（12.2 节的 `CONTEXT_MODE_EXTERNAL_MCP_NUDGE_EVERY=1` 是唯一被官方给出具体 token 量的数据点，约每次匹配调用 250 token；默认每 10 次重提一次的稀释使常态影响远小于该值）；以及无 hooks 平台上约四成操作不会被路由的隐性成本。节省一侧以索引缓存复用、批量合轮、stdout 收窄为主。折算维度上，官方数字与描述（例如 statusline 展示的金额与效率）给出直观口径；团队场景下可把「每千 token 单价 × 节省量」换算为可比指标。

17.3 故障排除表

按「症状—原因—处理」整理高频问题（出处均见对应章节）：

症状	常见原因	处理
Claude Code 中 `/plugin` 不被识别	版本低于 v1.0.33	`brew upgrade claude-code` 或 `npm update -g @anthropic-ai/claude-code`
工具可用但路由不生效；节省率停滞在约 60%	未启用 hooks（MCP-only 路径）或 hooks 文件位置不对	启用 hooks；确认文件在各自的 hooks 目录（如 `.github/hooks/` 而非 `.github/`）
`context-mode: command not found`	未做全局安装，或 PATH 未包含	`npm install -g context-mode`；`which context-mode` 验证路径
安装在 Linux 上报错	Node.js < 22.5（不支持）	升级 Node 或采用 Bun
CentOS/RHEL 上安装失败	better-sqlite3 需从源码编译	安装 2.6 节列出的编译链后启用工具集重装
`ctx_doctor` 某项显示未通过	运行时、hooks、FTS5 或注册版本异常	按 `ctx_doctor` 输出逐项修复；进一步信息用 15.2 节诊断脚本
Playwright 输出仍进上下文	调用未带 `filename` 参数	按 14.2 节工作流改写调用
检索质量不稳定	未带 `source`；查询次第分散	按 8.6 节四条调优动作修正
升级后 OpenCode/KiloCode 的 `ctx_` 工具消失	双注册（`plugin` 与旧 `mcp.context-mode` 并存）	`context-mode upgrade` 移除旧 MCP 条目（12.4 节）
某平台钩子无动作（不阻塞工具）	该平台钩子直接调用全局可执行文件而其版本仍旧	`npm install -g context-mode@latest`（12.4 节）
压缩后会话状态未恢复	平台评级为「部分」或「无」；或会话未以续接开启	对照 9.5 节矩阵核实平台；续接开启会话
误以为 `ctx_stats` 应删除内容	概念混淆	`ctx_stats` 只读；删除用 `ctx_purge(confirm: true)`

17.4 团队推广建议

把个人实践推广为团队默认配置，可行的路径为四步：

1. **试点**：选一个 MCP 用量大的项目组，两周记录 `ctx stats` 与压缩前后工作状态还原情况，形成内部对照数据。
2. **定型**：以「有 hooks 平台优先」为基线（10.1 节），无钩子平台统一教师文件并入仓库、并通过 `ctx doctor` 入驻检查。
3. **度量**：将 15.5 节的三层口径纳入例会与增量对比；避免以官方基准数字直接外推为团队收益承诺。
4. **安全**：将第 11 章的权限规则（`permissions`、`CTX_FETCH_STRICT`、落库脱敏）写入团队默认配置，并明确审批语义与宿主沙箱不得放松的要求。

17.5 资源与后续维护

本教程的依托资料均为官方载体（2026-10-30 核实）：GitHub 仓库 `mksglu/context-mode`（README、`docs/` 下的分平台指南、`BENCHMARK.md`）、npm 包 `context-mode`、`skills/context-mode/SKILL.md` 路由规则与模式库、官方 Discord 社区，以及 GitHub Issues 跟踪区。语义随版本演变，以官方文档为准：本教程基于 1.0.169 版本核实，后续版本的行为差异（尤其 hooks 协议与平台清单）应回到官方 README 对照。

附录

> 本附录按速查表体例整理正文的关键信息，均依据 2026-10-30 核实的官方文档编制，与正文互为索引；条目与正文表述不一致时，以正文为准。

附录 A 命令与工具速查

十一个 MCP 工具（模型调用）

工具	一句话职责	关键参数
`ctx_execute`	在沙箱运行一段代码，只回传 stdout	`language`（12 种语言之一）、`code`；超过 5 KB 且提供 `intent` 时启用意图过滤
`ctx_execute_file`	文件整块进入沙箱处理，回传结论	`path`、`language`、`code`；脚本内用预载变量 `FILE_CONTENT`
`ctx_batch_execute`	一次调用执行多命令与多检索	批量命令与查询集；`concurrency`（1 至 8，I/O 密集时）
`ctx_index`	按标题分块入库（FTS5 + BM25）	`path`（服务端读文件，方案首选）、`content`（仅限小段自编文本）、`source`（来源名，支持部分匹配）
`ctx_search`	检索知识库并回传智能片段	`queries`（数组，一次问完）、`source`、`contentType`（如 `code` / `prose`）
`ctx_fetch_and_index`	抓取 URL 转 markdown 入库并带 TTL 缓存	`url` 或 `requests` 数组、`ttl`（毫秒，默认 24 小时）、`force`（`true` 强制重抓）、`concurrency`
`ctx_stats`	查看节省、调用与会话统计（只读）	无
`ctx_doctor`	诊断运行时、hooks、FTS5 与注册版本	无
`ctx_upgrade`	升级、重建、迁移缓存并修 hooks	无
`ctx_purge`	永久删除全部已索引内容	`confirm: true`（必填）
`ctx_insight`	在浏览器打开托管分析页面	无

会话内命令（任何平台）：`ctx stats`、`ctx doctor`、`ctx index`、`ctx search`、`ctx upgrade`、`ctx purge`、`ctx insight`。

Claude Code **斜杠命令**：`/context-mode:ctx-stats`、`/context-mode:ctx-doctor`、`/context-mode:ctx-index`、`/context-mode:ctx-search`、`/context-mode:ctx-upgrade`、`/context-mode:ctx-purge`、`/context-mode:ctx-insight`。

终端命令：`context-mode doctor`、`context-mode index <路径> --source <来源名>`、`context-mode search "<查询词>" --source <来源名>`、`context-mode upgrade`、`context-mode insight`、`bash scripts/ctx-debug.sh`（完整诊断报告）。

附录 B 术语表

术语	定义
上下文窗口（context window）	模型单次推理可读取的全部信息总量，包含提示、历史、工具输出与自身回复
上下文压缩（compaction）	上下文接近上限时，代理丢弃或压缩较早对话以腾出空间的过程
MCP（Model Context Protocol）	模型与外部工具、数据源之间的标准连接协议
钩子（hook）	宿主在工具调用前后、会话开始结束、压缩前后等时点触发的拦截回调
路由（routing）	引导或强制代理把大输出操作转交沙箱工具执行的行为
教师文件	官方提供的路由指引文件（`AGENTS.md`、`GEMINI.md`、`QWEN.md`、`KIRO.md`、`SYSTEM.md`、`context-mode.mdc` 等），只靠提示词起作用，不能拦截调用
沙箱（sandbox）	context-mode 为每次代码执行启动的隔离子进程，原始数据在其内处理
stdout（标准输出）	程序打印到终端的文本流，沙箱产物进入上下文的唯一通道
沙箱工具	回传数据型 `ctx_` 工具的统称：`ctx_execute`、`ctx_execute_file`、`ctx_batch_execute`
知识库	本地 SQLite FTS5 索引库，承载 `ctx_index`、`ctx_search`、`ctx_fetch_and_index` 的内容
FTS5	SQLite 的全文检索扩展（Full-Text Search 版本 5），以倒排方式存储可检索文本
BM25	概率相关性排序算法，按词频、逆文档频率与文档长度归一对片段打分
Porter 词干化	把单词还原为词干的标准化算法；入库时应用，使不同词形互相命中
trigram（三元子串）	以三字符滑窗建立的子串索引，用于标识符等精确片段检索
RRF（倒数排名融合）	合并多个排序列表的算法；两路都靠前的结果浮到最上面
Levenshtein 编辑距离	两个字符串相差最少需要多少次增、删、改；用于检索前的拼写纠正
智能片段	围绕查询词命中位置截取窗口的结果展示方式（区别于从头截断）
intent（意图）	附在调用上的意图描述；输出超过 5 KB 时触发「入库 + 按意图检索」的自动降级
凭据透传	沙箱内 CLI 继承宿主环境变量与配置路径的机制，凭据明文不出现在对话
渐进节流	连续 `ctx_search` 调用被逐级降额直至拦截、并指向批量工具的机制
TTL	缓存有效期；默认 24 小时，`ttl: 0` 与 `force: true` 绕过
会话快照	压缩前生成的分优先级 XML 状态存档（约 2 KB 预算），存于 `session_resume` 表
会话指南（Session Guide）	压缩后重建的 19 栏结构化工作状态清单
事件	钩子捕获的结构化会话动作（文件、任务、git、错误、裁决等，共 23 类、分 P1 至 P4 四档）
子代理（subagent）	由主代理派生的助手会话；路由指引经由 PreToolUse 钩子自动注入
平台钉扎	在插件配置中固定 `CONTEXT_MODE_PLATFORM` 环境变量，避免多平台并装时识别混乱
MCP-only 模式	只注册 MCP 服务器、不启用 hooks 与斜杠命令的最小安装方式
落库脱敏	MCP 工具入参中凭据类键值在持久化前被替换为 `[REDACTED]` 的处理
ELv2	Elastic License 2.0；可用可改可分发，但不得包装为托管服务出售、不得移除许可声明

附录 C 逐平台安装速查

以下步骤为官方 README 安装一节的浓缩（2026-10-30 核实），每条路径的配置文件全文见仓库 `configs/` 目录。未特别说明时，通用前置为 `npm install -g context-mode`，安装后重启平台并在会话中输入 `ctx stats` 验证。

Claude Code（v1.0.33+）：会话内执行 `/plugin marketplace add mksglu/context-mode` 与 `/plugin install context-mode@context-mode`，重启；`/context-mode:ctx-doctor` 校验全 `[x]`。MCP-only 试用：`claude mcp add context-mode -- npx -y context-mode`。

Gemini CLI：`~/.gemini/settings.json` 同时写 `mcpServers.context-mode` 与四类 hooks（`BeforeTool`、`AfterTool`、`PreCompress`、`SessionStart`，命令为 `context-mode hook gemini-cli <事件名小写>`）；可选复制教师文件 `configs/gemini-cli/GEMINI.md`；`/mcp list` 中应显示 `Connected`。

VS Code Copilot（Copilot Chat v0.32+）：项目 `.vscode/mcp.json` 注册服务器；`.github/hooks/context-mode.json` 注册 `PreToolUse`、`PostToolUse`、`SessionStart`（`PreCompact` 的完整写法见 `configs/vscode-copilot/hooks.json`）；可选复制教师文件到 `.github/copilot-instructions.md`。

JetBrains Copilot（插件 v1.5.57+）：Settings > Tools > AI Assistant > Model Context Protocol 添加服务器（名称 `context-mode`、命令 `context-mode`）；hooks 文件与 VS Code Copilot 同位（`.github/hooks/context-mode.json`）。

GitHub Copilot CLI：插件一步装 `npm install -g context-mode` 后执行 `copilot plugin install mksglu/context-mode:configs/copilot-cli`；手动路径为 `copilot mcp add context-mode -- context-mode` 并在 `~/.copilot/hooks/context-mode.json` 写六个驼峰命名事件（或由 `context-mode upgrade` 代写 hooks 文件）；验证 `ctx stats` 与 `context-mode doctor`。

Cursor（市场插件待审，Issue #485）：本地装法为克隆仓库后 Windows 用 `robocopy . "$env:USERPROFILE\.cursor\plugins\local\context-mode" /MIR ...`（官方命令见 README），macOS/Linux 用 `ln -s`；手工装法为 `.cursor/mcp.json` + `.cursor/hooks.json`（`preToolUse`、`postToolUse`、`stop`，可加 `afterAgentResponse`）+ 复制 `configs/cursor/context-mode.mdc` 到 `.cursor/rules/`。

OpenCode：项目或全局 `opencode.json` 加 `"plugin": ["context-mode"]`；可选复制 `configs/opencode/AGENTS.md`；若配置同时残留 `mcp.context-mode` 旧键将注册出零个工具，用 `context-mode upgrade` 清理（v1.0.140+ 有 stderr 提示）。

KiloCode：与 OpenCode 同架构；配置文件为 `kilo.json`（初始化模板含 `$schema` 与 `"plugin": ["context-mode"]`）；教师文件同用 `configs/opencode/AGENTS.md`。

OpenClaw 网关（>2026.1.29）：克隆仓库后 `npm run install:openclaw`（读取 `OPENCLAW_STATE_DIR`，默认 `/openclaw`；自定义路径可用 `npm run install:openclaw -- /path/to/openclaw-state`）；安装器负责构建、`better-sqlite3` 重编、扩展注册与网关重启；文档见 `docs/adapters/openclaw.md`。

Codex CLI：插件路径执行 `codex plugin marketplace add mksglu/context-mode`，并在 `~/.codex/config.toml` 的 `[features]` 开启 `hooks` 与 `plugin_hooks` 后，在 Codex 提示时信任插件 hooks；存储不可写时以 `CONTEXT_MODE_DIR` 指到可写目录。手动路径：`[mcp_servers.context-mode]`（含 `CONTEXT_MODE_PLATFORM = "codex"` 环境变量）+ `$CODEX_HOME/hooks.json` 六事件 + 复制 `configs/codex/AGENTS.md`。

Kimi Code：`~/.kimi-code/mcp.json` 注册 MCP；`~/.kimi-code/config.toml` 以 `[[hooks]]` 表写六个事件（仅 `PreToolUse` 带 matcher，均 `timeout = 30`，命令形如 `context-mode hook kimi pretooluse`）；教师文件复用 `configs/codex/AGENTS.md`；文档见 `docs/adapters/kimi-code.md`。

Qwen Code：`~/.qwen/settings.json` 同时写 `mcpServers.context-mode` 与五事件 hooks（`PreToolUse` 的 matcher 覆盖 Shell、读文件、抓取与全部 `mcp__` 工具）；可选复制 `configs/qwen-code/QWEN.md`。

Antigravity IDE：`~/.gemini/antigravity/mcp_config.json` 注册 MCP（无 hooks）；手动复制教师文件 `configs/antigravity/GEMINI.md`。

Antigravity CLI（`agy`，要求 >= 1.0.7）：`npm install -g context-mode` 后执行 `agy plugin install https://github.com/mksglu/context-mode/tree/main/configs/antigravity-cli`；MCP-only 时改在 `~/.gemini/config/mcp_config.json`（注意与 IDE 路径不同）注册；卸载用 `agy plugin uninstall context-mode`。

Kiro：`.kiro/settings/mcp.json` 注册 MCP（或全局 `~/.kiro/settings/mcp.json`）；`.kiro/hooks/context-mode.json` 写 `preToolUse`（matcher 覆盖 `execute_bash`、`fs_read`、`@context-mode/` 工具与其余 MCP）与 `postToolUse`；教师文件 `configs/kiro/KIRO.md` 复制到项目根。

Zed：`~/.config/zed/settings.json`（Windows `%APPDATA%\Zed\settings.json`）的 `context_servers` 段注册 MCP（无 hooks）；复制教师文件 `configs/zed/AGENTS.md`。

Pi Coding Agent：`npm install -g context-mode` 后 `pi install npm:context-mode`（或把 `"npm:context-mode"` 写入 `~/.pi/agent/settings.json` 的 `packages`）；`~/.pi/agent/mcp.json`（或项目 `.pi/mcp.json`）注册 MCP。

OMP（Oh My Pi）：一步装 `omp plugin install context-mode` 后重启，`omp plugin list` 与 `omp plugin doctor` 显示 enabled；手动路径为在 `~/.omp/plugins` 目录 `bun add context-mode`（或 npm 等价）；插件自注册 MCP 于 `~/.omp/agent/mcp.json`（不覆盖既有条目，Issue #677）；MCP-only 路径另需复制 `configs/omp/SYSTEM.md`。

附录 D 环境变量速查

变量	默认	取值与含义
`CONTEXT_MODE_DIR`	各适配器默认根	存储根目录；绝对路径必填，`~` 不展开，空或空白视同未设置且由 `ctx_doctor` 明示；会话与统计位于 `<根>/sessions`，索引内容位于 `<根>/content`（v1.0.147+）
`CTX_FETCH_STRICT`	未设置	设为 `1` 时在默认网络封堵之上再禁回环与内网段；共享或 CI 环境部署时建议开启
`CONTEXT_MODE_EXTERNAL_MCP_NUDGE_EVERY`	`10`	外部 MCP 包裹提示的重提间隔（次）；范围 1 至 100，非法值回退为 10；设为 `1` 每次调用重提，约增加 250 token/次的开销

附录 E 反模式速查

来自官方 `SKILL.md` 反模式清单的高频条目（完整表见 13.6 节）：

1. Bash 中执行会产生大输出的命令（`curl`、`cat` 大文件、`npm test`、裸 `gh pr list`）。
2. 以上游截断代替下游过滤（`| head -20` 丢弃未查看的数据）。
3. Playwright 三件套不落盘（缺 `filename` 参数）或依赖 `browser_navigate` 自带的整页快照作检查依据。
4. 大数据或 MCP 响应走 `ctx_index(content:)`（上下文成本翻倍；应走 `path` 由服务端读盘，已在上下文中的数据直接使用）。
5. 指望 `ctx_stats` 进行删除（其只读；删除用 `ctx_purge(confirm: true)`）。
6. 检索分散为多次单查询调用（应合并进一个 `queries` 数组）。
7. 多来源检索不带 `source`（跨来源互相污染）。
8. 无 hooks 平台未放置教师文件，而把路由寄托于模型自觉（命中率约 60%）。